QWEN3.8-27B | ENGINE RESEARCH SERIES

SGLang 0.5.21 Research

বাংলা গবেষণা ডসিয়ার

Kaggle 4× NVIDIA L4 · Ada SM89

Source-backed experiments, correctness gates এবং measurable priorities

1500+ tok/s

Sustained aggregate accepted-output decode · research target

SGLang 0.5.21 | MODEL: Qwen3.8-27B | P0 / P1 / P2 / P3 | OFFLINE KAGGLE

VERSION 5.0 · 29 OCT 2026

Evidence-graded · URLs clickable

সূচিপত্র / Research sections

- Executive decision — first ten experiments
- A. Hardware and version-pinned deployment
- B. Exact baselines — benchmark arm vs production are different
- C. P0 correctness and leaderboard acceptance — FIRST, before faster kernels
- D. P1 highest payoff: decode critical path and DFlash efficiency
- E. P2 targeted engine and scheduling experiments
- F. P3 experimental research tracks — not default production
- G. Standard benchmark and correctness protocol
- H. Negative findings and discarded ideas
- I. Kaggle L4×4 adjacent sources (deployment ideas only)
- J. Primary sources — raw clickable URLs for Claude
- K. Version control and future AI-research integration
- L. V2.0 সংযোজন: AI Research 01, source-preserving integration এবং নতুন priorities
- M. Expanded experiment ledger for SGLang 0.5.21 and source-fork arms
- N. Critical quality research: SGLang INT4 W4A16, CUDA compute cache, and PPL
- O. Required speculative-state fixes: committed length, forced rejection, crash
- P. Cheapest speed experiment: DFlash W4×20 and width/stream co-optimization
- Q. NCCL/TP4 collective and source-level compute opportunities
- R. Long-prefill fairness, agent residency and the 90k–130k physics
- S. New high-upside R&D: EAGLE-4, single-card replicas and PP4
- T. Kaggle neighboring L4×4 evidence: preserve distinction between 14B/7B and 27B/90k
- U. Standard implementation, cheap GPU-run ordering and 1500+ admission gate
- V. Revision audit 01 — complete insight inventory and conflicts
- W. দ্বিতীয় AI গবেষণা — SGLang DFlash, Mamba concurrency ও NCCL-এর নতুন পরীক্ষা
- X. Version 3 — দ্বিতীয় AI-এর findings ও source coverage audit
- Y. Version 4 — তৃতীয় AI গবেষণা সমন্বয়: SGLang 0.5.21 / Qwen3.8-27B
- Z. v4 second audit — incoming AI insight-to-section ledger
- AA. Final integration v5: Claude E22 + তিন GPT + স্বাধীন গবেষণার পূর্ণ সমন্বয়

বাংলা Research Dossier — Version 5.0 | 29 October 2026 | FINAL FIVE-SOURCE INTEGRATION
Mission: Sustained aggregate accepted-output decode **1500+ tok/s**; within the 12-hour Kaggle budget, improve correctly submitted ARC answers, not only microbenchmark speed.

> **Scope:** Qwen3.8-27B + Kaggle 4×L4; SGLang 0.5.21 INT4+DFlash2 is the reference, **not an immutable configuration**. Quantization, parallelism, DFlash/EAGLE variants, Mamba strategy, CUDA graph and verified source backports may be changed, with a new evidence/

correctness row for each. Latest SGLang code is used **only as research/backport candidate**, not mislabeled as installed. Separate document treats vLLM 0.32.0. NVIDIA Team 1500+ claim is **user-specified target**, not re-verified here.

Evidence key. [LOCAL] = your stored EMS reports/GPU logs/notebook; **[UPSTREAM]** = someone else's linked primary-source issue/PR/benchmark; **[HYPOTHESIS]** = proposed optimization unmeasured on your 4×L4; **[P0]** = mandatory before production adoption. A measured public H200/GB10 speedup is never converted into an L4 prediction.

V4 CURRENT RESEARCH POLICY. 1500+ aggregate tok/s আমাদের target। অন্য GPU/engine fork-এর implementation হলো mining source: profiler ও patch-এর mechanism আলাদা করে 4×L4-এ port/test করা হবে। Current baseline stable reference, বাধ্যতামূলক স্থায়ী configuration নয়। Original V1+V2 sections preserved; new research and full tests added in W/X; follow **AA1 Version 5 final priority queue** first; Y1 remains the historical v4 campaign.

V5 FINAL RESEARCH NAVIGATOR. নতুন AI-এর দুইটি research segment বিশ্লেষণ করে **Y1**-এ নতুন highest-priority experiment matrix, **Y2–Y11**-এ implementation/negative tests এবং **Z**-এ complete insight-coverage audit যোগ হয়েছে। **Y1** আগে পড়ুন। V1/V2/V3-এর সব substantive sections ও primary URLs সংরক্ষিত; existing configs চলমান reference, freeze নয়।

VERSION 5 FINAL EXPERIMENT INDEX: প্রথমে **AA1** (new final priority order), তারপর **AA2–AA9/AA10** (Claude E22 raw NCCL evidence, source-backed code candidates, unresolved risks, five-study coverage ledger) পড়ুন। পুরোনো v1–v4-এর সব বিস্তারিত finding, raw URL এবং experiment priority history রাখা হয়েছে। Engine configs reference control, অপরিবর্তনীয় সিদ্ধান্ত নয়। 1500+ aggregate tok/s আমাদের user-specified research target।

Executive decision — first ten experiments

1. **S-P0-01:** resolve DFlash/INT4 **accuracy gap** and state correctness before any speed winner is trusted.
2. **S-P0-02:** source-verify #43567 stale Mamba checkpoint invalidation and #43559 allocation crash handling; do not mistake failed diagnostics for clean PASS.
3. **S-P1-01:** profile **critical-path** NCCL SHM all-reduce/all-gather and safely sweep channels/CTAs.
4. **S-P1-02:** DFlash width **4,5,6,7** × client concurrency/admitted streams; W4 nearly tied W6 despite shorter steps.
5. **S-P1-03:** 90k–130k cache-resident actual streams, Mamba slots and shared-prefix multi-turn scheduling.
6. **S-P1-04:** target VERIFY cost and per-step graph replay; keep source-correct state boundaries.

7. **S-P2-01:** overlapping long prefill vs decode; avoid cold refills/restarts/tool-turn eviction.
8. **S-P2-02:** lm_head BF16, Marlin INT4, SM89 Triton GDN kernel and TP vocab gather profiling.
9. **S-P3-01:** ReplaySSM **only** after demonstrating `extra_buffer_lazy` compatibility; naive `no_buffer` lost prefix cache.
10. **S-P3-02:** PP4 / source-level communication overlap only after P1 profiles show real critical-path headroom.

A. Hardware and version-pinned deployment

Element	Locked value / consequences
GPU and host	4× NVIDIA L4, 24 GB/GPU, Ada SM89; PCIe only, no NVLink/P2P guarantee. 48 vCPU, ~197GB RAM, ~92GB `/dev/shm`.
Execution	Kaggle offline runtime, 12h maximum, installed wheelhouses & kernels attached datasets, load+warmup <~10min.
Serving	SGLang 0.5.21 isolated venv, cu13-compatible user-space, Python 3.12 notebook. Default TP4 NCCL SHM; FlashInfer attention, **Triton GDN**.
Model weights	RedHat INT4 `pack-quantized` compressed-tensors W4A16 g128; lm_head/MTP BF16 unquantized components; verify ignore list and actual loaded tensor schema.
DFlash	BF16 DFlash2 drafter; `dflash_config.target_layer_ids == [5,19,33,47,61]`, `num_target_layers == 64` guard from E17 planning.
Hybrid network	48 Gated DeltaNet linear-attention layers, 16 full attention; Mamba checkpoint and partial attention KV must stay aligned.

Provenance: `Q27_optimize_vLLM032_SGLang_2026-10-29.zip` → `sglang/README_sglang.md`, `sglang/results/ems_report.md`, original server logs, E14 payload code; `Q27_best_vLLM_SGLang_notebooks_2026-10-29.zip` → production E15B Duck notebook.

B. Exact baselines — benchmark arm vs production are different

B1. EMS fastest measured speed arm (`dflash_w6_s20`)

```
python -m sglang.launch_server \
  --model-path "$INT4_MODEL" \
  --tp-size 4 \
  --context-length 155648 \
  --kv-cache-dtype fp8_e4m3 \
  --mamba-ssm-dtype float16 \
  --mamba-radix-cache-strategy extra_buffer_lazy \
  --max-mamba-cache-size 84 \
```

```
    --speculative-algorithm DFLASH \
    --speculative-draft-model-path "$DFLASH_BF16" \
    --speculative-draft-model-quantization unquant \
    --speculative-num-draft-tokens 6 \
    --max-running-requests 21 \
    --chunked-prefill-size 4096 \
    --mem-fraction-static 0.83 \
    --attention-backend flashinfer \
    --linear-attn-backend triton \
    --disable-flashinfer-autotune
```

Illustrative core flags: check the exact generated `EMS` flags, `sglang_server_ems_dflash_w6_s20.log` and SGLang 0.5.21 `--help` before copy/paste. In particular, the installed CLI may use a slightly different spelling for the linear-attention flag (`--linear-attn-backend` versus `--linear-attention-backend`); the engine-specific source builder remains authoritative. Other flags include page-size 1, LPM scheduler, seed and `--min-free-slots-delay 1`.

This arm requests **20 client streams**, server max-running **21**, and Mamba slots **84**. 90k XL actual running was **7**, not 20.

B2. Production E15B Duck configuration

Production notebook `Q27-XHIGH-E15B_EVAL120_50K45K_S16_R3K2_DUCK_SGLANG.ipynb` runs **16 client streams**, and E15B E14 baseline uses Mamba cache ~**68** with `--max-running-requests 17`, decode CUDA graphs 1–17 and the `extra_buffer_lazy` path. It is **not** the EMS S20/84-slot benchmark. The original early `e11_flags()` text mentions DFlash8; the adopted later override sets DFlash **width 6**. Use the effective final launch argv, not the early helper, in all comparisons.

B3. Baseline throughput / acceptance / latency

SGLang EMS arm	S	8k tok/s	40k tok/s	XL ~90k tok/s	XL actually running	8k/40k accepted/ step
DFlash W6 (best speed)	20	**597**	**536**	**364**	**7**	4.17 / 4.21
DFlash W6 reference	16	583	521	370	8	4.21 / 4.11
DFlash W4	16	**592**	**526**	363	**9**	3.30 / 3.31
DFlash W8	16	573	477	363	7	5.00 / 4.47
MTP $(3,1,4)$	16	478	402	296	11	2.88 / 2.94
Target-only	16	405	354	275	15	1 / 1
DFlash W6 + PCIe-IPC	16	230	202	233	8	4.32 / 3.91

Source: stored EMS `sglang/results/ems_report.md`. Note `tok/s` is steady decode after prefill settles, **not final competition solve/hour**. EMS classified W6/W4 as BOUNDARY target-only parity (no large-margin flips in tested probes), **not exact cross-engine long-ARC accuracy**. W6 S24 was capped/OOM, not tested as working 24-stream XL.

B4. Why target 1500+ requires structural changes

The 90k W6-S20 arm at **364 tok/s** requires **4.12×** improvement. At measured 7 active × ~4.22 accepted/step, 1500 requires ~**19.7 ms/step**. Its observed XL step ~81.3 ms; blindly increasing DFlash width cannot make up this gap. At 20 active × ~4.21 accepted/step, 1500 corresponds to ~**56.1 ms/step**, but 20 resident unique 90k prompts currently do not fit. Thus two major levers must combine: **faster critical-path + more meaningful resident streams/shared-state reuse**.

C. P0 correctness and leaderboard acceptance — FIRST, before faster kernels

S-P0-01 — Explain ~6–7 leaderboard-point deficit, not just token parity

Your pack reports the same Poetiq method's SGLang result roughly **6–7 points below vLLM FP8** in earlier submissions; this is **not conclusively attributable to DFlash**. Other plausible causes: INT4 vs FP8 quantization and calibrated KV, speculative target/verify state, different engine precision, context scheduling, cache stale restoration, final-drain/coverage, and sampling variance. Control **one axis at a time**:

1. **Same INT4 checkpoint, same SGLang**: target-only with cache OFF versus DFlash6 on/off with cache ON; frozen prompts and deterministic greedy.
2. **Same task and budget across engines**: SGLang INT4 target-only vs vLLM 0.32 FP8 target-only, report score/coverage separately (token exact cross-quantization is not required).
3. **FP8 vs INT4 model difference**: if an FP8 SGLang target-only arm is available, use it to isolate model quantization from engine.
4. **Source/precision:** lm_head actually BF16 and excluded from INT4; actual GDN state dtype; FP8 KV scales checkpoint ↔ runtime; true selected kernels and all model warnings.
5. **Downstream:** paired 24–64 ARC held-out tasks, matched wall/token budgets, McNemar on discordant solves, final output JSON shape/coverage, multi-turn tool-call validity. A correct output after 1024-token probe does not certify 60k/90k reasoning.

Upstream DFlash wrong-answer evidence: issue #36548 reported serial 0/100 wrong, concurrent DFlash8 111/304 wrong, target-only 0/304 on a different GB10/DGX environment. Reducing width, using fp32 SSM, disabling graphs improved but did not entirely fix their issue. This is an existence proof, **not your measured bug rate**.

Source: https://github.com/sgl-project/sglang/issues/36548

S-P0-02 — E17 #43567 stale Mamba checkpoint path

A previous E17 short GPU run with **four Mamba slots** hit `AssertionError: Can not alloc mamba cache` in all four relevant arms **before the intended optional lazy allocation / stale restore chain**; counters `optional_alloc_failed=0`. This proves an allocation diagnostic crash, **not** #43567 corruption absent or present in production.

#43567 mechanism [UPSTREAM]: lazy optional `alloc(1)` fails, pending slot unavailable, persistent keep slot overwritten in place, old `mamba_last_track_seqlen` left valid; a checkpoint can be donated at an obsolete boundary, then restored for another request sharing prefix → wrong recurrent state/logits. Two-hunk fix invalidates length to `None` on failed optional allocation in ordinary and speculative preparation. Verify patch SHA and exact installed source state.

#43559 separate mechanism [UPSTREAM + local 4-slot crash class]: Mamba donation allocation can assert with no available/evictable slots while old prefix lock remains held; its upstream fix unlocks Mamba-related prefix reference and retries allocation. **Never install #43559 alone without checkpoint-invalidation safety.**

Cheapest decisive diagnostic: unpatched vs exact #43567 backport, forced optional allocation fail **after valid checkpoint seeding**, ordinary and DFlash W6; same request correlation: `alloc_failed -> stale donation -> matched restore -> full-vocab KL > own A/A tolerance`; patched arm correct; protect server from #43559 crash. Then **natural** 68/84-slot 16/20-stream pressure to learn production exposure rate. Missing link = `INCONCLUSIVE`, not PASS. No claim that these bugs explain a 26-point score gap.

Source: https://github.com/sgl-project/sglang/pull/43567 ; https://github.com/sgl-project/sglang/pull/43559

S-P0-03 — Other speculative correctness risks

- **#43029:** hybrid speculative committed-length boundary/state mismatch fix in later SGLang; not assumed present on 0.5.21. Test accepted/committed length and force-reject verify at track boundaries. https://github.com/sgl-project/sglang/pull/43029
- **#43514:** DFlash acceptance collapse / degenerate punctuation output; record accepted/step, empty/repetition, thinking/grammar/tool calls. https://github.com/sgl-project/sglang/issues/43514
- **#42917:** compilation cache stale artifacts and W4A16 numerical gap; fresh dedicated `CUDA_CACHE_PATH` and SGLang kernel cache, store hashes, compare first-token logits. https://github.com/sgl-project/sglang/issues/42917
- **#45151:** speculative MTP cache slot duplication / starvation backport only if the **MTP** branch is explored, not automatically required for DFlash6. https://github.com/sgl-project/sglang/pull/45151
- **#38009:** DFlash greedy output unstable across restarts; match source SHA, include multi-boot A/A and separate true near-tie noise. https://github.com/sgl-project/sglang/issues/38009

S-P0-04 — Fail-closed performance certificate

Require exact model checkpoint hash, drafter `dflash_config.target_layer_ids=[5,19,33,47,61]` and 64 target layers, all TP ranks loaded, full-vocab or top-margin reference with complete evidence, cold/warm cached-depth expected, actual incoming user request isolated, no empty output, no stalled progress, no out-of-memory, no new allocation assert. Keep per-TP-rank trace but don't fabricate per-request causal chain by summing unrelated counters. Test `ignored_eos`, `max_new_tokens`, client abort and five repeated cache cycles to detect slot leaks and wrong restore.

D. P1 highest payoff: decode critical path and DFlash efficiency

S-P1-01 — NCCL SHM TP4 collective tuning

LOCAL profile: approximately 50.9 ms all-reduce plus 8.1 ms all-gather of a 116 ms step, while weight read is ~16–22 ms. These are **summed profiling durations**; inspect Nsight/PyTorch trace for overlap before calling them additive critical-path latency. Default SHM path, ~2 channels; ~344µs for ~1MB all-reduce in one captured microbenchmark. Forcing P2P reportedly ~5.5× slower, FlashInfer PCIe-IPC full model 0.39× (SGLang W6 8k 583→230, 40k 521→202).

GPU matrix (each is separate server env):

```
# N0: default NCCL SHM
# N1: low channels
NCCL_MIN_CTAS=1 NCCL_MAX_CTAS=1
# N2: repeat measured default neighborhood
NCCL_MIN_CTAS=2 NCCL_MAX_CTAS=2
# N3/N4: if supported by installed NCCL wheel
NCCL_MIN_CTAS=4 NCCL_MAX_CTAS=4
NCCL_MIN_CTAS=8 NCCL_MAX_CTAS=8
# N5/N6: isolate algorithms, do not mix with CTA sweep
NCCL_ALGO=Ring
NCCL_ALGO=Tree
# Optional new arm only after microbench
NCCL_NTHREADS=512
```

First actual E14 microbenchmark (1MB / 4 ranks / graph); then same W6 S16+S20 at 8k/40k and 90k. Measure collective **critical path** in CUDA graph, active SMs, verification step, context, accepted/step, model TPS and request time. Reject fast NCCL microbench if GPU model slower. Avoid PCIe-IPC repeat unless a verifiably changed backend warrants it.

Docs: https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/env.html

S-P1-02 — DFlash width × admitted concurrency optimized together

Why: W4 S16 592/526 vs W6 S16 583/521 at 8k/40k; W4 step 89.2/100.9ms vs W6 115.7/127.3ms. W8 acceptance grows to 5.00/4.47 but **throughput falls** 573/477. More drafted tokens are not free.

Experiment	Width	Client streams	Reason
W0	6	16, 20	anchor; same code/weights
W1	4	16, 20	shorter verify, sometimes faster
W2	5	16, 20	unmeasured intermediate
W3	7	16, 20	unmeasured acceptance-latency optimum
W4	best 2 widths	actual XL resident streams	90k meaningful comparison

Example **single-axis delta**:

```
--speculative-algorithm DFLASH \
--speculative-num-draft-tokens 5
```

For every width record accepted final-output tokens/draft step, target verify wall ms, draft kernel time, graph replay sizes 1..N, actual active requests, Mamba slot requirements and total accepted TPS. **Do not use W8 as a repeat winning hypothesis**; its measured XL speed tied/lagged. Run full quality gate after shortlist.

S-P1-03 — Cache capacity and agent-level concurrency

At XL W6 S20, reported cache capacity ~779k and **7 active**, W4 S16 **9 active**; target-only has more cache but slower pure decode. No simple 20×90k unique-prompt residency exists with current settings. The production 16-stream configuration has different Mamba slots than the 20-stream benchmark. Never reduce `--max-mamba-cache-size` just to admit more KV: Mamba allocation asserts and false cache reuse are real risks.

Experiments: Mamba slots 68/84/85 and matching max-running 17/21, memory-static 0.80/0.83/0.86 under strict health+capacity; shared 4 tasks×4 agents vs unique 16/20 tasks; 50k/90k/130k; prefix-hit, evictable vs available slots, in-use Mamba, queueing, Mamba alloc failure and #43567 stale candidate counts. **Measure full agent work per wall second**, not only steady-state decode.

S-P1-04 — Verify latency, TP gather and graph capture

Target-only S16 step **39.5/45.1 ms** at 8k/40k; DFlash W6 **115.7/127.3 ms**, W4 **89.2/100.9 ms**. Verification/drafter and state cost is a first-order target. Profile per-layer GDN Triton verify, INT4 Marlin GEMM, BF16 lm_head and 248k-vocab sampling, CUDA graph captured batches 1..17/21, PTX/JIT effects, NCCL critical path. Optimize only a measured dominant kernel. Any altered verify order/state gather requires committed-token and forced-reject parity.

Relevant upstream: https://github.com/sgl-project/sglang/issues/36548 ; https://github.com/sgl-project/sglang/pull/43029

E. P2 targeted engine and scheduling experiments

S-P2-01 — Reduce expensive re-prefill and prefill/decode interference

Your shared-prefix Duck turns can append 2k feedback while carrying 50–130k history. If prefix checkpoint becomes evicted or inaccessible, an entire task-turn is prefilling again and decode streams lose throughput. Run separate D1 resident decode, D2 shared-prefix agent turns and D3 unique 50/90/130k mixed prefills. Sweep chunked prefill 2048/4096/8192 and decode admission, keeping numeric/kernel flags fixed. Observe per-step iteration gap, queuing, cache-hit depth and old checkpoint validity. A warm loop scoring 536 steady decode may still be poor in a cold multi-turn real agent session.

Contrasting public report (vLLM, not automatically SGLang): https://github.com/vllm-project/vllm/issues/54919. SGLang uses a different scheduler; only actual L4 measurements decide.

S-P2-02 — INT4 target weight/FP8 KV/checkpoint numeric quality

RedHat config has compressed-tensors W4A16 and calibrated FP8 KV. Earlier SGLang E17 raw traces recorded non-unit runtime scales for 16 full-attention layers (K~0.0277–0.0513, V~0.0214–0.2539) across arms. That disputes a claim that every runtime scale is 1.0, **but checkpoint-vs-runtime exact tensor equality remains unverified**. Capture weights/scales under each TP rank; test fp8_e4m3 vs bf16 KV at reduced context only if memory fits, target-only oracle; then restore production context. A dtype change may lose speed, capacity or accuracy.

S-P2-03 — L4 GDN backend constraints and kernel decisions

Do **not** use FlashInfer GDN accelerated backend on SM89; existing SGLang engine-matrix source check says SM90+ required. CuTe DSL GDN prefill compiled for SM100+, and decode/verify fallback to Triton on L4; toggling it without effective kernel change is a no-op. Keep `--attention-backend flashinfer` for **full attention**, `--linear-attn-backend triton` for **GDN**; separate these two backends. New source-level kernel patch must compile in offline wheelhouse and show real activation on sm89. Details in LOCAL EMS notes.

S-P2-04 — MTP branch and safety patch #45151 (lower priority)

SGLang MTP `(3,1,4)` reached **478/402/296** vs DFlash W6 **583/521/370** on S16; existing throughput inferior, so only keep as diagnostic/safety fallback. Source https://github.com/sgl-project/sglang/pull/45151 addresses redundant MTP state allocation/starving cache in a different branch. Avoid merging it into DFlash production without need. If adopting MTP or new MTP head path, validate this PR, slot leaks and same-oracle parity.

S-P2-05 — Runtime scheduling and generated output loss

Monitor in-flight HTTP, actual generation token counter, CPU tool calls, deadline drain, `stragglers`, and final `submission.json` completeness. Do not rely solely on `/health` for server stalls. On vLLM E21 harness, cross-half callback races and incomplete comparison gates were previously found; the same general discipline applies here if combining server matrix arms in one notebook. A long 12-hour competition benefits more from 5% fewer late/invalid outputs than from an uncertified 5% kernel uplift.

F. P3 experimental research tracks — not default production

S-P3-01 — ReplaySSM with extra-buffer compatibility (NO blind enable)

Issue **#37834** specifically tested Qwen3.8-27B and found `--enable-linear-replayssm` forces `--mamba-radix-cache-strategy no_buffer` in affected SGLang versions. In one saturated shared-prefix test on a Blackwell TP1 GPU, prefix hits **71.91%→29.23%**, output **134.5→93.4 tok/s** (−30.6%), and TTFT worsened. Another unsaturated load showed TTFT 226→1063ms; throughput was pinned by arrival. **Not your L4 measurements.** The regression occurs because checkpoint donated beyond the shared prompt boundary isn't reusable by sibling requests.

Decision: do not blindly add `--enable-linear-replayssm` to 0.5.21 production `extra_buffer_lazy`. Test only if upstream patch restores correct extra_buffer/cache checkpoint and ring write cursor (`replayssm_write_pos`) across donation; apply exact-source backport with 6h soak/cold-warm parity and multi-turn prefix-hit measurement. Standalone ReplaySSM 1.2–1.5× GDN microkernel claim ≠ end-to-end 1500× result.

Sources: https://github.com/sgl-project/sglang/issues/37834 ; https://github.com/sgl-project/sglang/issues/28511 ; https://github.com/sgl-project/sglang/pull/28695

S-P3-02 — Pipeline-parallel PP4 to avoid TP4 per-layer collective

Candidate architecture TP1+PP4 could remove frequent PCIe all-reduces, but introduces inter-stage activations and pipeline bubbles; GDN state, INT4 compressed tensors and DFlash draft placement may not be supported. First **target-only 8k** startup and exact model load, then 40k/90k, then DFlash width4/6 if supported, no fallback mislabeling. Reject if stage imbalance or startup >10min. Source/issue https://github.com/sgl-project/sglang/issues/43885

S-P3-03 — Host-staged or overlapped all-reduce experiment

Source PR https://github.com/sgl-project/sglang/pull/39605 describes host-staged path (source-pack note: TP2 only/open). Not a TP4 copy-paste fix. Read code and confirm CUDA events, SHM buffering, stable ordering, TP4 collective equivalence, offline compilation. Existing PCIe-IPC was 0.39×: a different implementation must be demonstrably faster on TP4, not merely another P2P guess.

S-P3-04 — Upgrade/backport study without changing track

SGLang 0.5.22 and later include fixes #43567/#43559/#43029 under different commit windows; compare pinned 0.5.21 source diffs and test one hunk at a time, with wheel SHA and reproducible rollback. The research target remains 0.5.21; a proposed upgrade is **separate engine migration**, never called a 0.5.21 flag-only optimization.

G. Standard benchmark and correctness protocol

Required three workloads

- **D1: resident decode:** 8k/40k/90k/130k prompts prefilled first; no cold prefills, comparable actual running streams, 20min elapsed accepted-output TPS.
- **D2: shared-prefix long-agent:** 4 ARC tasks ×4 streams, repeated +2k tool feedback, up to 4k generation, compute cache hits including generated tails, mixed-time output TPS, task-level correctness.
- **D3: distinct prompts:** 16–24 different long tasks, measure admitted concurrency, Mamba slot contention, KV memory, preemption, EOS and final drain.

Primary metrics

```
# proposed schema; None fields MUST be measured before claiming GO
record = {
```

```
  'engine':'sglang-0.5.21', 'arm':'dflash_w5_s20',
  'model_sha':'...', 'drafter_sha':'...', 'wheel_sha':'...',
  'requested_streams':20, 'running_p50':None,
  'context_p50':None, 'cached_prefix_tokens':None,
  'kv_pool_tokens':None, 'mamba_size':None,
  'mamba_available':None, 'mamba_evictable':None,
  'accepted_output_tokens':None, 'elapsed_seconds':None,
  'aggregate_tps':None, 'accept_tokens_per_step':None,
  'target_verify_step_ms':None,
  'collective_critical_path_ms':None,
  'cold_prefill_seconds':None, 'failures':None,
  'stale_restore_count':None,'startup_seconds':None,
  'target_only_gate_pass':False, 'arc_nonregression_pass':False,
}
```

Fail-closed rules

NO-GO if: missing target-only reference, insufficient logprob/cold evidence, high-confidence greedy mismatch not explained by own A/A, multi-request state crossover, stale cached Mamba checkpoint, `AssertionError: Can not alloc mamba cache`, repetitive punctuation/empty output, response length/margin evidence missing, Mamba slot leak, unexpected backend fallback, startup >600s, or end-to-end solve coverage degradation. A handful of near-tie flips can be `BOUNDARY`, not exact parity; promote only after paired ARC outcomes.

Recommended staged run order

Stage	Experiments	Required proof
S0 (CPU)	E14/E15 source and patches, source SHA, DFlash target_layer_ids, installed flags, offline kernels	no launch errors
S1 (bounded 8k)	W4/W5/W6/W7 × 16 streams, single-arm 3/3 content	baseline & correctness
S2 (40k)	top 2 widths × S16/S20, NCCL N0–N4	accepted TPS, critical-path improvement
S3 (90k–130k)	selected arms; shared and distinct; Mamba 68/84, 16/20	actual concurrent, no crash
S4 (state correctness)	forced rejection, ordinary/spec cache boundary, 2-source multi-turn; check #43567/#43559	causal chain & clean patch
S5 (paired ARC)	24–64 tasks, same wall/token budget, seed controls	accuracy non-regression
S6 (production-like)	12h-equivalent Duck run, tool calls and final drain	correctly submitted outputs/hour

H. Negative findings and discarded ideas

Candidate	Why not in P1
FlashInfer PCIe-IPC custom all-reduce	W6 s16 583→230 / 521→202 tok/s, ~0.39×.

Candidate	Why not in P1
Forced NCCL P2P	Microbench ~5.5× slower in available topology; no NVLink guarantee.
DFlash W8 everywhere	Larger accepted length but lower 8k/40k TPS; no XL gain.
W6 S24 blindly	CAPPED/OOM, not real 24-stream XL.
MTP $(3,1,4)$ as direct speed upgrade	478/402 vs 583/521 W6 S16.
FlashInfer GDN on sm89	Requires SM90+; engine refuses.
CuTe DSL GDN verify on sm89	No effective supported verify kernel; prefill cutoff SM100+.
ReplaySSM + `no_buffer`	Known shared-prefix hit/TTFT regression.
Four-slot #43567 natural test unchanged	Actually #43559-class allocation assertion before trigger.
Any proposed FP8 KV scale=1.0 blanket claim	E17 raw traces found non-unit scales; checkpoint binding not yet fully proven.

I. Kaggle L4×4 adjacent sources (deployment ideas only)

- ARC Prize 2026: https://www.kaggle.com/competitions/arc-prize-2026-arc-agi-2
- ARC Prize 2025: https://www.kaggle.com/competitions/arc-prize-2025
- Gemma 4 Developer Agent: https://www.kaggle.com/competitions/gemma-4-developer-agent
- Konwinski Prize: https://www.kaggle.com/competitions/konwinski-prize
- 72B AWQ 4×L4 notebook: https://www.kaggle.com/code/abdullahmeda/load-72b-awq-model-using-vllm-on-l4-x4

No listed Kaggle experiment is misrepresented as Qwen3.8-27B SGLang 90k/1500+ benchmark proof. Other teams' speed reports need exact architecture, request mix, prefill included and actual stream count before they can be compared.

J. Primary sources — raw clickable URLs for Claude

[S01] SGLang repo source: https://github.com/sgl-project/sglang/tree/v0.5.21

[S02] Qwen3.8 concurrent DFlash wrong answers: https://github.com/sgl-project/sglang/issues/36548

[S03] Stale lazy Mamba checkpoint fix: https://github.com/sgl-project/sglang/pull/43567

[S04] Prefix lock + Mamba allocation crash fix: https://github.com/sgl-project/sglang/pull/43559

[S05] Spec accepted/committed boundary fix: https://github.com/sgl-project/sglang/pull/43029

[S06] DFlash acceptance collapse: https://github.com/sgl-project/sglang/issues/43514

[S07] CUDA/kernel cache & W4A16 perplexity issue: https://github.com/sgl-project/sglang/issues/42917

[S08] Speculative MTP cache-slot fix: https://github.com/sgl-project/sglang/pull/45151

[S09] DFlash greedy restart instability: https://github.com/sgl-project/sglang/issues/38009

[S10] Qwen3.8 ReplaySSM breaks prefix reuse: https://github.com/sgl-project/sglang/issues/37834

[S11] ReplaySSM design/performance: https://github.com/sgl-project/sglang/issues/28511

[S12] ReplaySSM implementation PR: https://github.com/sgl-project/sglang/pull/28695

[S13] Host-staged communication proposal: https://github.com/sgl-project/sglang/pull/39605

[S14] SGLang pipeline-parallel support question: https://github.com/sgl-project/sglang/issues/43885

[S15] NCCL environment variables: https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/env.html

[S16] NCCL performance / troubleshooting: https://github.com/NVIDIA/nccl/blob/master/docs/userguide/source/troubleshooting/performance_and_tuning.rst

[S17] SGLang registered Mamba-cache KL regression: https://raw.githubusercontent.com/sgl-project/sglang/main/test/registered/radix_cache/test_mamba_cache_kl.py

[S18] vLLM SM89 Qwen hybrid prefill interference (contrast): https://github.com/vllm-project/vllm/issues/54919

[S19] SGLang release notes: https://github.com/sgl-project/sglang/releases

[S20] SM89 single-card Qwen3.8 DFlash MMQ/GEMM benchmark (llama.cpp, not drop-in): https://github.com/hanxiao/Qwen3.8-27B-UD-Q4_K_XL-L4

[S21] Full Kaggle Qwen discussion reference supplied previously: https://www.kaggle.com/competitions/arc-prize-2026-arc-agi-2/discussion/750546

[S22] SGLang standalone docs: https://docs.sglang.ai/

K. Version control and future AI-research integration

This **Version 2.0** report stays engine-specific. Integrate later AI research as new `S-P*` experiment rows with original URL, asserted speed gain, source hardware/method, pinned-code availability, risk, precise patch/flag, low-cost test, measured outcome, and revision history. Never replace a failed local run with a generic upstream quote or silently promote a speculative patch to production. Target **1500+ sustained** remains open; prioritize **evidence-backed multiple interacting bottlenecks** rather than a single miracle CUDA flag.

L. V2.0 সংযোজন: AI Research 01, source-preserving integration এবং নতুন priorities

Intake: `Pasted text(9).txt`, দুইটি দীর্ঘ research write-up (29 October 2026)। V1-এর সব পূর্ববর্তী findings ও failed tests সংরক্ষিত; এই অধ্যায়ে newly discovered candidates যোগ করা হয়েছে। SGLang 0.5.21 হলো reference branch; **একই configuration ধরে রাখার বাধ্যবাধকতা নেই**। DFlash width, quantized checkpoint, SSM dtype, prefix strategy, all-reduce, PP, TP, new spec drafter, fork/backport—সব পরীক্ষা করা যাবে; কিন্তু নতুন build-কে নতুন engine arm এবং নতুন correctness baseline হিসেবে লগ করতে হবে। 1500+ sustained accepted-output tok/s আমাদের লক্ষ্য; NVIDIA Team-এর প্রমাণ অনুসন্ধান এই scope-এ নয়।

L1. Revised high-priority conclusions

- **P0: SGLang INT4 quality parity before deployment.** Upstream issue #42917 initially showed a large W4A16 PPL regression; subsequent author follow-up (as reported in the incoming research) identified stale **CUDA driver compute cache** as cause. That distinction is crucial: do not rewrite INT4 kernels before fresh-driver-cache same-input PPL. Your own 6–7 LB gap remains unsolved. Issue: https://github.com/sgl-project/sglang/issues/42917
- **P0: DFlash speculative recurrent state and tool/grammar truncation.** #36548/#35150/#43514 and committed-length PR #43029; only target-only parity on long, concurrent, tool-using turns is sufficient to rule out a tested error class. Source: https://github.com/sgl-project/sglang/pull/43029
- **P1: DFlash W4 ×20 streams is the cheapest untested neighborhood** of the currently measured W6 S20 winner and W4 S16. Compare W4/W5/W6/W7, actual running count and per-step accepted output, not just draft acceptance.
- **P1: TP4 collective critical path (NCCL SHM).** Local profile claim ~50.9 ms all-reduce + 8.1 ms all-gather during ~116ms step (raw Nsight critical path absent in this intake). Tune CTA/channels only after timestamped real trace; PCIe-IPC already regressed 0.4×.
- **P1/P2: 2k/4k/8k chunked prefill, decode fairness, Mamba cache slots, token-identical tool-turn reuse.** Long cold prefills cost throughput even if resident decode microbench unchanged.
- **P2/P3: alternative SGLang speculative EAGLE-4 with source-version migration, Merlin-inspired kernel ideas and TP1/PP4.** They are high-upside but **not** flags already supported in SGLang v0.5.21.

M. Expanded experiment ledger for SGLang 0.5.21 and source-fork arms

ID	Rank	Exact lever / arm	Test evidence required	Reject reason
S21-01	P0	Fresh isolated CUDA **driver** compute cache, target-only PPL	checkpoint SHA, prompt token IDs, cold repeated logits/PPL vs vLLM 0.32 INT4	PPL remains far worse or unstable

ID	Rank	Exact lever / arm	Test evidence required	Reject reason
S21-02	P0	#43567 stale slot invalidate + #43559 allocation retry/lock handling	alloc fail→donate→restore witness, patched and unpatched; no slot leak	Wrong restore or assertion
S21-03	P0	#43029 committed-length grammar/ truncation fix	forced speculative acceptance truncated by parser/grammar; compare committed GDN state	Any phantom checkpoint/boundary
S21-04	P0	DFlash concurrent/ forced-rejection control #36548/#35150	target-only serial vs concurrent, DFlash serial vs concurrent; long 2k→60k	High-margin target change, repeated garbage/tools
S21-05	P1	DFlash **W4 S20**, W5 S16/S20, W6 controls, W7	same 8k/40k/90k, step latency, accepted/ verify and actual active	XL active collapses or net tok/s not better
S21-06	P1	NCCL `MIN_CTAS/ MAX_CTAS` controlled sweep	initial real critical path, then single-variable A/B, TP4 SHM	e2e slower, CUDA starvation, unsupported topology
S21-07	P1	2048/4096/8192 prefill chunk tuning	16 resident decode +1 cold90k; tool-turn wall-tok/s	decoded stream starvation / TTFT worse
S21-08	P1	Mamba slots 68/84 and shared-prefix residency	actual allocation/ eviction and cached generated tail	OOM or corruption / no more active streams
S21-09	P1/P2	CUDA graph decode/ verify capture sizes	selected graphs, kernel launch gap, memory and startup time	graph OOM/incorrect verification
S21-10	P2	GDN `in_proj_ba` split-K from Merlin idea	SM89 tensor dimensions, numerical oracle, same INT4 target	packed mapping wrong or no e2e gain
S21-11	P2	Draft/verify no-sync and FlashInfer plan caching	host/GPU synchronization profile, speculative acceptance fixed	race/changed output or no performance
S21-12	P2	DFlash width×batch adaptive policy	existing native supports dynamic? otherwise isolated per-server arms	unsupported patch / expensive draft
S21-13	P3	EAGLE-4 27B drafter via compatible **fork**	sm89/VRAM/precision, 1/8/16+ acceptance, source license	incompatible engine, state drift, >600s load

ID	Rank	Exact lever / arm	Test evidence required	Reject reason
S21-14	P3	TP1/PP4 with INT4 target-only	one-GPU/stage memory fit + GDN state, then DFlash	bubble / too few live 90k streams
S21-15	P3	ReplaySSM preserving `extra_buffer_lazy`	compare cached prefix hits and long rejected-state replay	forced `no_buffer` drops hit/TTFT, wrong ring cursor
S21-16	P3	alternate ~16GB 27B Merlin checkpoint / host embeddings	SGLang quantized loader mapping verified; one L4 boot	incompatible packed tensor names, wrong outputs, memory
S21-17	P2	NCCL share-memory/ CPU NUMA and TP vocab gather	CUDA/NCCL timestamps, TP4 output after accepted step	no critical-path gain, 0.4× PCIe-IPC repeats

`GO for diagnostic` ≠ `GO for production`. The experiment registry includes high-risk options because the user explicitly allows changing configuration repeatedly; the adoption gate remains strict.

N. Critical quality research: SGLang INT4 W4A16, CUDA compute cache, and PPL

N1. Original upstream report #42917: before and after

Issue #42917 tests `dbirks/Qwen3.8-27B-W4A16-AutoRound` on **RTX5090 SM120** using 20 WikiText2 chunks; test set 20,460 scored positions; target-only, BF16 KV and radix disabled. The initial issue text reports SGLang 0.5.21 **PPL 9.9767, top1 69.1%**, vLLM 0.30 **PPL 6.0528, top1 92.9%** on same checkpoint; the INT4 kernel looked suspect. This is **another checkpoint (symmetric W4A16)** and **another GPU**; not proof RedHat checkpoint is inaccurate on L4. Source: https://github.com/sgl-project/sglang/issues/42917

Incoming AI Research 01 reports a subsequent author resolution (10 Oct): stale `~/.cache/sglang/nv` CUDA driver compute-cache caused wrong code; fresh cache gives **PPL 6.2784, top1 93.6%, bit-identical repeats**, versus stale PPL ~10.85–10.91 and ~70% top1. These final-comment numbers are **research-supplied**, not independently re-executed here; mark them as author follow-up, not original issue body. Critical nuance: clearing only Triton/JIT caches did not eliminate the contamination in that report. A fresh new driver cache directory is safer than destructive deletion of user caches.

N2. Correct driver-cache isolation, not a merely cosmetic SGLANG_CACHE_DIR

```
# Before importing torch/sglang and before spawning SGLang workers:
export CUDA_CACHE_PATH="/kaggle/working/q27_cuda_cache_${ARM_ID}"
export CUDA_CACHE_DISABLE=0
export TRITON_CACHE_DIR="/kaggle/working/q27_triton_${ARM_ID}"
mkdir -p "$CUDA_CACHE_PATH" "$TRITON_CACHE_DIR"
```

```
# Optional separate SGLang cache; do not assume it replaces CUDA_CACHE_PATH.
export SGLANG_CACHE_DIR="/kaggle/working/q27_sglang_${ARM_ID}"
```

Preflight: log the resolved environment in every scheduler/GPU-rank process and source hash of compiled kernels. Fresh cache has cold-JIT/startup cost; compare warmed second runs with identical caches and log launch ≤600 seconds. If `CUDA_CACHE_PATH` is overridden by container/driver or not writable, classify test `INVALID`, not `CLEAN_CACHE_PASS`. Avoid mixing stale one arm and clean other arm with changed kernel selection/flags.

Minimal PPL matrix (same 20k target positions and token IDs): SGLang INT4 target-only clean driver cache, repeat clean-cache launch, vLLM 0.32 same INT4 target-only; compare full input NLL/PPL, logprob/next-token KL and FP8 KV scale evidence. Then add DFlash6 while preserving the target weights. Test FP8 reference separately for model quantization. Use matching tokenizer, chat template and prompt token IDs; do not extrapolate issue #42917's SM120 PPL to SM89.

N3. Leaderboard-gap causal decomposition

Control	Same	Changed	Meaning
SGL INT4 target-only vs SGL INT4 DFlash	weight, engine	speculation only	DFlash state/sampling concern
SGL INT4 target-only vs vLLM INT4 target-only	target checkpoint	engine implementation	loaded weights/GDN/kernel/ attention parity
vLLM INT4 target-only vs vLLM FP8 target-only	engine and input	quantization	target quality difference
SGL FP8 target-only vs vLLM FP8 target-only (if supported)	FP8 checkpoint	engine	numerical implementation
Same engine after clean driver cache	checkpoint/config	code cache state	compiler-cache contamination
Same solver with equal wall and token budgets	task, rubric	engine or quant	true solved-task impact

Separate **valid submission coverage**, conditional solve accuracy, early EOS, malformed tool, long-context length, sampling temperature and deadline drain. A 6–7 leaderboard-point difference could be orchestration as well as model; no single unrelated upstream PPL issue proves the cause.

O. Required speculative-state fixes: committed length, forced rejection, crash

O1. #43029: accepted length is not always committed length

PR #43029 corrected speculative Mamba checkpoint boundary tracking where a grammar/tool output path verified tokens but committed a **shorter** prefix. Old code could checkpoint at an uncommitted boundary, later serving incorrect GDN state. The incoming research dates its merge 10 Oct 2026, after v0.5.21. Link: https://github.com/sgl-project/sglang/pull/43029

Backport protocol: verify hunk absent in pinned `schedule_batch.py` / `batch_result_processor.py`; find actual `kv_committed_len`, `accepted_len`, `planned_pos` and checkpoint tracking. Inject a grammar truncation such that `accepted > committed`, then matched new request hits the stored prefix and compare target-only logits. Long forced-reject runs with 8/16 concurrent check state restoration. Don't mark a patch safe just because unit tests pass on non-grammar traffic.

O2. #43567 and #43559 must not be conflated

- #43567 prevents stale lazy checkpoint donated after optional pending-slot allocation failure. Source: https://github.com/sgl-project/sglang/pull/43567
- #43559 fixes separate `AssertionError: Can not alloc mamba cache` when an unfinished request's prefix lock blocks donation allocation; it is not a blanket output correctness patch. Source: https://github.com/sgl-project/sglang/pull/43559
- Your earlier 4-slot E17 tests in four arms stopped at allocation assertion with `optional_alloc_failed=0`; they do **not** show #43567 absent. A forced-fail branch and paired patched/unpatched event chain can diagnose target effect, then natural 68/84 slots captures production exposure. Actual decode request path must trace fail→in-place overwrite→donation→restore→KL.
- Use controlled `CUDA_CACHE_PATH` and whole-process restart between patched/unpatched arms; do not reuse rank workers or globally disable safety checks.

O3. DFlash correctness cannot be inferred from a clean 1k probe

Sources: https://github.com/sgl-project/sglang/issues/36548 ; https://github.com/sgl-project/sglang/issues/35150 ; https://github.com/sgl-project/sglang/issues/43514 ; https://github.com/sgl-project/sglang/issues/38009

Independent test design: target-only serial/c16, DFlash serial/c16, 8k/40k/90k prompts, exact frozen token IDs, greedy and temperature=1.0 separate arms, acceptance/rejection traces, tool parser + grammar, long 30k–90k generated reasoning (as feasible), early-EOS/malformed-output checks. A large-margin first-divergence is a correctness signal; low-margin stochastic divergence alone isn't proof. Compare actual **committed** prefix position, not merely draft proposal length. Issue auto-close is not the same as fix.

P. Cheapest speed experiment: DFlash W4×20 and width/stream co-optimization

Saved EMS shows:

DFlash	Client streams	8k	40k	Step latency at 40k	90k XL
W6	16	583	521	127.3ms	370 (8 active)
W6	20	597	536	157.1ms	364 (7 active)
W4	16	592	526	100.9ms	363 (9 active)

DFlash	Client streams	8k	40k	Step latency at 40k	90k XL
W8	16	573	477	149.9ms	363 (7 active)

W4×20 was not measured in previous reports. Keep model, drafter, KV FP8, SSM dtype, prefix strategy, page size, generated budget and sampling identical; only `--speculative-num-draft-tokens 4` and client streams=20 change from a named reference. Set SGLang max-running guard and Mamba cache slots consistently; **do not** compare W4 S20 vs W6 S16 and attribute the entire difference to width.

```
# W4 S20 proposed experimental delta
--speculative-algorithm DFLASH \
--speculative-num-draft-tokens 4 \
--max-running-requests 21
# Client scheduling: 20 requested streams; log ACTUAL running p50/p90.
```

Predictive accounting, not promised speed: `accepted TPS ≈ effective_running × mean_accepted_per_verify / step_seconds`. If W4 accepts ~3.3 and has 20 active, 900 TPS requires ~73ms critical step; W6 accepting ~4.2 requires ~93ms. At XL only 7–9 active the demand becomes much harder. First 8k+40k controlled speed screen, then 90k shared and distinct prefix, then full ARC parity. For W5 and W7 always compare identical S16/S20 and log accepted lengths (draft overhead can offset acceptance).

Q. NCCL/TP4 collective and source-level compute opportunities

Q1. Critical-path profiling before CTA tuning

Your supplied profiler summary says a ~116ms decode/verify step contains **50.9ms all-reduce** and **8.1ms all-gather**. These are **cumulative operation durations**, not yet verified CUDA critical-path elapsed times: overlapping kernels may make naive 59ms sum inaccurate. Obtain Nsight event trace with rank0 + cross-rank timings, message size, kernel CTA count and synchronization boundaries, verify no real NVLink/P2P. The safe default remains NCCL SHM, not FlashInfer PCIe-IPC (measured ~0.39×).

Experimental matrix, one launch each, with exact original environment preserved except variable:

Arm	Env delta	What to measure
N0	No CTA override	Baseline all-reduce critical path
N1	`NCCL_MIN_CTAS=1; NCCL_MAX_CTAS=1`	Small-message bandwidth vs SM occupancy
N2	`NCCL_MIN_CTAS=2; NCCL_MAX_CTAS=2`	Two-CTA reference
N3	`NCCL_MIN_CTAS=4; NCCL_MAX_CTAS=4`	Four-CTA utilization

Arm	Env delta	What to measure
N4	`NCCL_MIN_CTAS=8; NCCL_MAX_CTAS=8`	Aggressive test only if GPU resources permit
N5	`NCCL_ALGO=Ring` or `Tree`	Supported algorithm only; no P2P forcing

Research suggested `NCCL_NTHREADS=512`; treat it as an independent test **only after confirming actual NCCL version accepts it** and reading warnings. Prefer CTAs first. Measure 1MB all-reduce microbench AND whole engine 40k/90k; e2e objective supersedes microbench. NVIDIA official: https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/env.html ; https://github.com/NVIDIA/nccl/blob/master/docs/userguide/source/troubleshooting/performance_and_tuning.rst

Q2. GDN, Marlin, sampling and metadata

Merlin (vLLM 0.32 fork) advertises a GDN `in_proj_ba` **split-K** kernel, host FlashInfer decode-plan caching, uniform multi-token CUDA graphs, second MTP draft no-sync and RecoverSSM. These are **inspiration/source-code targets**, not drop-in SGLang v0.5.21 flags. Reimplement only after pinned-source shape and attention-GDN compatibility study. Every GDN projection change must verify packed `in_proj_qkvz`/`in_proj_ba` mapping and recurrent-state numerics. Source: https://github.com/halt95/merlin-engine ; https://github.com/halt95/qwen38-27b-3090s

Qwen3.8 full-attention head_dim256 and TP4 sampling require measuring FlashInfer attention kernel, FP8 KV-scale application, TP vocab all-gather, lm_head (BF16), and CPU/GPU sync. L4 SM89 cannot use SM90+-only FlashInfer GDN acceleration. A custom source patch is justified only after the saved Nsight critical path proves that precise kernel dominates.

R. Long-prefill fairness, agent residency and the 90k–130k physics

R1. 2048 vs 4096 vs 8192 chunks

The incoming research cites SGLang Qwen3.8 cookbook guidance that large 8192-token prefill chunks may starve concurrent decode, suggesting ~2048 chunks. This is a **mixed workload efficiency** candidate, not pure decode-kernel acceleration. Source documentation: https://docs.sglang.ai/ ; specific cookbook link not furnished in pasted text, **exact page unverified**.

Three workloads per arm:

1. D1: 16 resident 40k–90k decode streams **without** new prefill; measure true kernel TPS.
2. D2: same decodes while one 90k cold input prefill enters; collect decode throughput collapse, queue, TTFT, prefill seconds and all-rank active count.
3. D3: real ARC multi-turn trace, identical tool feedback, shared vs unique prefix, score accepted output / all wall time including tool wait and final drain.

A smaller chunk may increase kernel launches/host overhead and slow cold prefill while improving fairness. Reject if whole-run correctly submitted answers/hour worsens; report tradeoff rather than only TTFT.

R2. 90k concurrent occupancy and theoretical KV lower bound

FP8 full-attention KV TP4/GPU ≈8192 bytes/token. Thus 20×90k unique full contexts require ~14.7GiB/GPU attention KV **before** model, drafter, GDN/Mamba, graphs and memory fragmentation; 20×130k ~21.3GiB/GPU, 24×130k ~25.6GiB/GPU and is not physically GPU-resident on a 24GB L4. Your SGLang W6 S20 XL test has ~779,192-token cache capacity and **7 effective running**; do not claim 20 live 90k streams. Shared prefixes can reduce logical duplicates but not arbitrarily zero attention cost.

L4 theoretical 300GB/s bandwidth gives a very rough ~49ms pass to read 14.7GB KV if all data is streamed once. This ignores attention tiling, warp sharing, speculative verification, model-weight reads, PCIe collectives, and actual bandwidth; not a measured lower-bound on your exact CUDA attention kernel. It indicates the importance of residency and GDN/cache scheduling.

R3. Cache strategy and ReplaySSM regression

Upstream Qwen3.8 issue #37834 reports that enabling ReplaySSM under affected SGLang configurations forced `no_buffer`, reducing cache hits **71.91→29.23%** and output **134.5→93.4 tok/s** on another GPU. Keep your current `extra_buffer_lazy` unless a tested integration preserves cached state. ReplaySSM ring write/commit can cause recurrent state contamination on deep generations. Before turning it on, trace cached prefixes and long rejected drafts. Sources: https://github.com/sgl-project/sglang/issues/37834 ; https://github.com/sgl-project/sglang/issues/28511 ; https://github.com/sgl-project/sglang/pull/28695

S. New high-upside R&D: EAGLE-4, single-card replicas and PP4

S1. EAGLE-4 as a distinct forked-engine track

Official EAGLE-4 Qwen3.8-27B model card at **4×B300 TP4 c1 T=0**: vanilla 240.5, MTP 534.5, DFlash2 672.6, EAGLE4 **863.8 tok/s**. The drafter adaptively changes draft tree based on workload and hardware, aiming to keep speculation profitable at high concurrency. **These numbers are not sm89/L4 performance and not your sustained 90k results.** Sources: https://github.com/eagle-hq/EAGLE-4 ; https://huggingface.co/EAGLE-Corp/Qwen3.8-27B-EAGLE4

EAGLE project includes an `eagle-harness` plugin using a patched SGLang dev version; it is **not already installed in v0.5.21**. Requirements: inspect license and competition permissibility, check CUDA/cu13 sm89 kernels, use same target checkpoint/quantization, measure draft model extra VRAM and 600s startup. First run one 8k target-only reference, then 8k EAGLE with forced rejection, then c8/c16 40k and 90k. Stop if one engine restart/prefix change triggers wrong GDN state. It is a serious alternative to DFlash, not an automatic replacement.

S2. TP1×4 replication with a different 24GB-fit checkpoint

Merlin reports 4×**CMP 170HX** (64GB/GPU, no P2P), 8 agents/16k: TP4 371, TP2×2 523, TP1×4 873 tok/s. **Never label that as 4×L4.** Source: https://github.com/halt95/merlin-engine . A separate 27B INT4 checkpoint (~16GiB with INT4 lm_head/MTP, host embeddings) is documented fitting one **24GB RTX3090**, and therefore might fit one L4 in a new engine/checkpoint: https://huggingface.co/halt95/Qwen3.8-27B-W4A16-Merlin . RedHat INT4 does not have the same head layout/VRAM profile.

Testing under SGLang requires first verifying compressed-tensors packed checkpoint mapping, host-embedding support, model load, DFlash drafter memory and **one L4 90k+ generation reserve**. If single-card active concurrency is 1–2, four replicas only support 4–8 unique 90k streams; no 20-stream miracle. A version-pinned vLLM Merlin-fork feasibility run may be cheaper than porting host embeddings into SGLang, but should be reported in the separate vLLM PDF as well.

S3. TP1/PP4 pipeline strategy

TP4 has per-layer PCIe collectives; PP4 can remove them at the price of activation transfers, pipeline bubbles and DFlash stage placement. Candidate `--tp-size 1 --pp-size 4` spelling must be verified against the **installed** 0.5.21 CLI. Sequence: target-only short context → split placement and KV/Mamba allocations → 40k/90k → DFlash if supported → full tool/grammar correctness. Drop if startup >600s, output drift or smaller number of actual long requests. Related PP issue: https://github.com/sgl-project/sglang/issues/43885

T. Kaggle neighboring L4×4 evidence: preserve distinction between 14B/7B and 27B/90k

- **AIMO-2 fifth-place 14B AWQ:** external AI report cites 4×L4 at 9 streams, 13.5k context: LMDeploy 304, SGLang 263, vLLM 238 aggregate tok/s. It is a harness-comparison template, not Qwen3.8 throughput evidence. Specific winning write-up URL was not included in pasted research; **source deep URL pending verification**.
- **AIMO-2 7B AWQ:** reported SGLang batch32 ~1,103 tok/s 4×L4, different weight count/context; proof only that hardware can have >1k on a different problem.
- **AIMO-2 repeatable throughput notebook:** test grid and 51m duration, not proof that this Qwen 27B engine fits more than seven 90k active streams.
- **Gemma 4 Developer Agent:** 4×L4 graph warmup and KV memory sizing can OOM before the first real request; reserve capture memory.
- **ARC-AGI-3 FP8 repack:** faster checkpoint loading if model weights unchanged is not better model accuracy; treat load time, not decode speed.
- **ARC-AGI-2 Kaggle** offline dataset/wheelhouse/12h limit must be reproduced; any special CUDA kernels must be staged and compiled for SM89.

Raw landing URLs: https://www.kaggle.com/competitions/ai-mathematical-olympiad-progress-prize-2 ; https://www.kaggle.com/competitions/gemma-4-developer-agent ; https://www.kaggle.com/competitions/arc-prize-2026-arc-agi-2 ; https://www.kaggle.com/competitions/arc-

prize-2026-arc-agi-3 ; https://www.kaggle.com/code/abdullahmeda/load-72b-awq-model-using-vllm-on-l4-x4 . These are category/resource links; don't claim an exact AIMO benchmark is checked until its actual notebook/write-up URL is obtained.

U. Standard implementation, cheap GPU-run ordering and 1500+ admission gate

U1. Minimal-cost screening in order

Order	What to run	Why first	GPU proof required
0	Offline source hashes + clean CUDA driver-cache + PPL / INT4 target-only	Prior observed INT4 LB gap; cheap and safety critical	stable same-input logits/PPL
1	DFlash W4 S20 alongside W6 S20	Only width variable, adjacent measured configuration	8/40/90k accepted tok/s, effective running
2	DFlash W5 S16/S20, W7 if W5 helps	Fill untested width gap	per-step draft/verify ratios
3	NCCL CTAs 1/2/4 after profiler	TP4 collective overhead suspected	real critical-path step decrease
4	Prefill chunks 2048/4096/8192	Improve long-turn fairness and cache	mixed output/wall throughput
5	#43029 plus #43567/#43559 patch matrix	Guard against silent speculative state corruption	causal trace + no crashes
6	CUDA graph and FlashInfer plan-cache backport	Profiling-specific optimization	same outputs, graph coverage
7	Long XL admission, memory slots, shared-prefix task affinity	Increase actual concurrency	90k/130k stable, no OOM
8	EAGLE-4 fork/PP4/TP1×4 feasibility	Structural ceiling research	SM89, accuracy, startup and memory fit

The user may choose any priority if profiling justifies it; all experiments have run IDs, exact source/build SHA and revert paths. A change that is good only at 8k but loses at 90k cannot be promoted as a 1500+ long-context solution.

U2. Exact metric schema and practical acceptance

```
row = {
 'engine_build_sha':None,'checkpoint_sha':None,'drafter_sha':None,
 'arm':'S21-05_W4_s20','cuda_cache_path':None,
 'requested_streams':20,'actual_running_p50':None,
 'context_p50':None,'unique_prefix_fraction':None,
 'cached_generated_tokens':None,'mamba_size':None,'mamba_evictable':None,
 'kv_pool_tokens':None,'graph_mode_effective':None,
 'accepted_output_tokens':None,'wall_seconds':None,
```

```
  'aggregate_output_tps':None,'step_ms_p50':None,
  'draft_ms':None,'verify_ms':None,'nccl_critical_path_ms':None,
  'prefill_interference_drop_pct':None,'malformed_tool_calls':None,
  'early_eos_count':None,'oom_count':None,'stale_restore_count':None,
  'target_only_gate':'NOT_RUN','paired_arc_gate':'NOT_RUN',
  'startup_sec':None,'offline_wheel_ok':False,
 }
```

For 1500 research target, label **decode-only resident** vs **mixed agent wall** vs **correctly submitted ARC**. A successful fast resident decode test is not proof 12h output coverage. Real 90k models currently ~7 active W6, not 20; physical KV memory + dynamic admission must be measured. Score+coverage is the actual adoption metric, not standalone accepted-tokens-per-verify.

U3. Fail-closed execution gates

Any new arm is **NO-GO** if source patch provenance missing, attempted flag unsupported, JIT wheel requires internet during Kaggle scored run, 600s startup exceeded, model differs silently from declared checkpoint, Mamba stale donation/restore, CUDA kernel invalid on SM89, serious target-only logprob drift, grammar/tool-call corruption, repeated punctuation, output cap/truncation unreported, missing cold/warm controls, or final submission unit coverage falls. Every `INCOMPLETE` and `NOT_EXERCISED` is explicitly non-PASS. Use synthetic negative tests in addition to pre-existing GPT harness tests.

V. Revision audit 01 — complete insight inventory and conflicts

Four-pass audit required after each future PDF revision: (A) input insight-to-section mapping, (B) full URL/provenance audit, (C) PDF-render/text/link integrity, (D) benchmark/negative-test coverage. No short-summary-only replacement of the original narrative.

First AI's research point	SGLang PDF location	Inclusion/qualification
Merlin 4×CMP TP1 873 and 24GB alternative model	S2	Included; CMP per GPU 64GB, not L4
Merlin five kernel patches + host embedding	Q2 / S2	Included; SGLang port is new engineering
EAGLE4 863.8 B300 and adaptive drafting	S1	Included; patched engine; license and SM89 unverified
Upstream W4A16 issue #42917 original bad PPL	N1	Included with checkpoint/GPU caveats
Upstream stale CUDA driver-cache resolution	N1/N2	Included; original and final evidence separated
`CUDA_CACHE_PATH` rather than only Triton cache	N2	Included; process-env verification gate
DFlash W4 × S20 missing A/B	P	Included; cheapest performance experiment

First AI's research point	SGLang PDF location	Inclusion/qualification
SGL DFlash #36548/#35150/#43514	O3	Included; real concurrency/long reasoning tests
#43029 committed-length checkpoint fix	O1	Included; tool/grammar truncation test
#43567 vs #43559 allocation correctness	O2	Included; distinct mechanisms
NCCL CTA plus optional `NTHREADS=512`	Q1	Included; only if version-supported
SGL prefill 2048 vs 4096 vs 8192	R1	Included; source cookbook exact URL pending
90k 7 active and KV capacity arithmetic	R2	Included; no 20-active false claims
ReplaySSM `no_buffer` regression	R3	Included; not an easy 1.5×
Alternative TP1/PP4, no-P2P	S2/S3	Included; model checkpoint and memory gate
AIMO2/Gemma4/ARC3 competition lessons	T	Included; deep notebook URL pending
FlashInfer sm90+ GDN kernels	Q2	Included; not viable sm89 direct drop-in
Long agent 12h wall TPS and submitted correctness	U	Included; three denominators separate
vLLM-only BF16 GDN / FULL graph / SM89 FP8	Separate vLLM v2 PDF	Correctly engine-separated; not omitted
Flash-Next vs 27B dense	Scope note	Excluded as different model, no false 27B gain

No silent source substitution: In the incoming AI material many evidence labels only read “GitHub”, “Kaggle” or “Hugging Face”, without their actual target URL. The PDFs replace those placeholders with **explicit known primary issue/repo links** where grounded; where exact deep link could not be confirmed, the limitation is explicitly marked. Any later AI research can add the original source URL to this ledger.

Additional raw URLs for Claude

- https://github.com/sgl-project/sglang/issues/42917
- https://github.com/sgl-project/sglang/pull/43029
- https://github.com/sgl-project/sglang/pull/43567
- https://github.com/sgl-project/sglang/pull/43559
- https://github.com/sgl-project/sglang/issues/36548
- https://github.com/sgl-project/sglang/issues/35150
- https://github.com/sgl-project/sglang/issues/43514
- https://github.com/sgl-project/sglang/issues/37834

- https://github.com/sgl-project/sglang/issues/28511
- https://github.com/sgl-project/sglang/pull/28695
- https://github.com/eagle-hq/EAGLE-4
- https://huggingface.co/EAGLE-Corp/Qwen3.8-27B-EAGLE4
- https://github.com/halt95/merlin-engine
- https://huggingface.co/halt95/Qwen3.8-27B-W4A16-Merlin
- https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/env.html
- https://www.kaggle.com/competitions/ai-mathematical-olympiad-progress-prize-2
- https://www.kaggle.com/competitions/arc-prize-2026-arc-agi-2

W. দ্বিতীয় AI গবেষণা — SGLang DFlash, Mamba concurrency ও NCCL-এর নতুন পরীক্ষা

গবেষণার নীতি: আমাদের লক্ষ্য **1500+ sustained aggregate accepted tok/s**। SGLang 0.5.21 INT4+DFlash6 হলো measured starting configuration, স্থায়ী constraint নয়। নতুন checkpoint, quantization, speculation, TP/PP, CUDA graph, Mamba strategy এবং targeted source backport পরীক্ষা করা যাবে। অন্য GPU বা serving fork-এ প্রকাশিত সফল experiment থেকে **বাস্তব code-level method, kernel design, scheduling ও benchmark protocol** নিতে হবে এবং Kaggle 4×L4-এ পরীক্ষা করতে হবে। কোন technique port করা যায় সেটাই প্রশ্ন; শুধু original hardware ভিন্ন বলে সম্ভাবনা বাদ দেওয়া নয়। প্রতিটি arm-এ correctness, memory residency, offline deployment ও ১০ মিনিটের startup gate প্রযোজ্য।

`Pasted text(10).txt`-এ **দুটি দীর্ঘ research narrative এবং URL appendix** আছে। দুটির source-backed findings নিচে আলাদা করে যোগ করেছি। V1/V2-এর সব আগের আলোচনা রেখে দেওয়া হয়েছে।

W1. SGLang-এর নতুন experiment priority—প্রথম দশটি

Priority	Experiment	প্রথম প্রমাণ/GO gate
P0 `S32-01`	DFlash width4 × requested20, W6×20 control	Same 8k/40k/90k prompts; actual admitted, accepted/step, output TPS
P0 `S32-02`	Mamba slots silent concurrency cap #36889	Resolved cap, free/evictable/in-use, actual concurrent
P0 `S32-03`	#41736/#41737/#43029 selective correctness fixes	Cross-request tap isolation, all-NaN, committed-length negative tests
P0 `S32-04`	Cold prefill বনাম resident decode interference	Warm/cold 19+1, 16+4; chunks 2048/4096/8192
P0 `S32-05`	NCCL SHM collective critical-path sweep	Real engine step সময়, not only synthetic all-reduce
P1 `S32-06`	Mamba radix split → 0-hit #22935	Exact repeat, 1/16/64/512 suffix mutation, correct checkpoint
P1 `S32-07`	INT4 target-only বনাম DFlash quality isolation	Greedy margin/state + paired ARC result

Priority	Experiment	প্রথম প্রমাণ/GO gate
P1 `S32-08`	v0.5.22-এর selective backports	Each fix separately, source SHA, regression tests
P2 `S32-09`	#39605 no-P2P staged all-reduce	Distinct algorithm-এর source port, PCIe-IPC repetition নয়
P2 `S32-10`	GDN/MTP/host embedding ও alternative draft kernels	Source extraction, memory-fit, accuracy first

Priority ≠ production permission। কোনো perf winner correctness fail করলে `NO-GO`; কোনো prefix path exercise না হলে `NOT_EXERCISED`। 8k peak, 90k sustained এবং 12-hour agent solves/hour পৃথক score।

W2. `S32-01` — DFlash W4 × 20: সবচেয়ে সস্তা নতুন configuration cell

আপনার EMS matrix-এর কাছাকাছি arms: **W4 S16 = 592/526/363** এবং **W6 S20 = 597/536/364 tok/s** (8k/40k/XL)। `W4 S20` পরীক্ষা হয়নি। এটি সরাসরি নতুন factorial cell: একই INT4 RedHat weights, DFlash2 BF16 drafter, FP8 KV, `extra_buffer_lazy`, context, warm-prefix এবং generation budget রেখে **শুধু width** বদলান। অন্য arm-এ client streams বদলাবেন।

```
--speculative-algorithm DFLASH \
--speculative-num-draft-tokens 4
# Client-side target 20 streams notebook-এর actual scheduler variable-এ দিন।
```

Matrix: W4-S16, W4-S20, W6-S16, W6-S20; এরপর curve ভালো হলে W5/W7। Record actual running p50/p95, KV+Mamba slots, draft acceptance, target accepted tokens, verification-step ms, all-reduce time ও aggregate accepted TPS। বিশেষ করে **90k W6-S20-এ ~7 actual running**—requested 20-কে real 20 বলবেন না। Width কমিয়ে verify সস্তা হলেও accepted tokens এত কমতে পারে যে speed একই থাকে; দুটি হিসাব ছাড়া winner নয়। DFlash output corruption বা malformed tool-call পেলে STOP।

W3. `S32-02` — Mamba-state cache slots-এর লুকানো admission cap

SGLang issue #36889-এ একটি hybrid KDA+DFlash configuration `--max-running-requests=8` গ্রহণ করেও মাত্র **2 actual concurrent** চালাচ্ছিল; কারণ 10 Mamba slots এবং প্রতিটি active request-এর বেশি slots প্রয়োজন। Source-এর সংশোধিত allocation-এ নির্দিষ্ট workload-এ প্রায় **75% throughput বৃদ্ধি** রিপোর্ট হয়েছে। আমাদের কাজ হলো এই **diagnostic method** Qwen3.8-এর resolved state-pool-এ প্রয়োগ করা। Nominal extra_buffer≈5, extra_buffer_lazy≈4, no_buffer≈3 slots/request—এসব recipe-derived starting values; actual wheel-এ startup log ও allocator state decisive।

সকল benchmark arms-এ log করুন: requested/engine resolved max-running, derived `max_mamba_cache_size`, slot cost per request, available/evictable/in-use, state dtype bytes, `mem_fraction_static`, KV capacity, queued/retracted requests এবং runtime `effective_running`। Existing 20-stream speed arm-এর Mamba84 configuration বনাম physically safe alternative slot counts-এ A/B; intentionally low-capacity negative test দিয়ে monitor সত্যিই cap ধরছে কি না দেখুন।

Safety: 24GB GPU-তে Mamba pool বাড়ালে KV ও CUDA graph memory কমতে পারে। `extra_buffer_lazy`-তে #43567 stale donation-এর আলাদা correctness risk আছে এবং #43559 allocation assert-ও পৃথক; slots বাড়ালেই দুটি fix হয় না। Mamba cap দায়ী কি না, KV বা scheduler cap-এর evidence বাদ দিয়ে সিদ্ধান্ত নেবেন না।

https://github.com/sgl-project/sglang/issues/36889

https://github.com/sgl-project/sglang/blob/main/docs/cookbook/autoregressive/Qwen/Qwen3.8-27B.mdx

https://github.com/sgl-project/sglang/pull/43567

https://github.com/sgl-project/sglang/pull/43559

W4. `S32-03/08` — তিনটি precise correctness backport: #41736, #41737, #43029

SGLang 0.5.22 release notes-এ DFlash/GDN correctness-এর **তিনটি ভিন্ন mechanism** উল্লেখ আছে। 0.5.21 source hash ও upstream diff না মিলিয়ে এগুলোকে already fixed বলা যাবে না। Full engine upgrade-এর বদলে একেকটি source overlay ও negative regression test দিয়ে backport পরীক্ষা করুন।

PR	আসল ঝুঁকি	Patch test
#41736	DFlash grouped-convolution tap আগের/পাশের request block-এর data বা NaN/Inf মিশিয়ে দিতে পারে	প্রতিটি request block-এ ভিন্ন sentinel, boundary tap, 1/8/16 concurrency; neighbouring output অপরিবর্তিত
#41737	DFlash2 all-NaN score row-এ greedy selector range-এর বাইরে যেতে পারে	All-NaN/no-valid-candidate input inject; safe fallback, no invalid index
#43029	Grammar/speculative verify-এর accepted length সবসময় **committed length** নয়; ভুল Mamba checkpoint boundary	Forced grammar truncation, rejected draft, actual committed position ও target-only next-token parity

আপনার throughput ranking-এর পাশাপাশি #36548 concurrent wrong answers ও #38009 greedy thinking divergence-এর context-এ এই tests বিশেষ প্রাসঙ্গিক। তবে upstream report থেকে **আমাদের 6–7 leaderboard-point gap-এর কারণ নিশ্চিত** করা যাবে না। Short 1k smoke যথেষ্ট নয়: 30k+ generation, high-margin first divergence, tool-call grammar, forced rejection এবং pool-pressure negative tests রাখুন।

https://github.com/sgl-project/sglang/releases/tag/v0.5.22

https://github.com/sgl-project/sglang/pull/41736

https://github.com/sgl-project/sglang/pull/41737

https://github.com/sgl-project/sglang/pull/43029

https://github.com/sgl-project/sglang/issues/36548

https://github.com/sgl-project/sglang/issues/38009

https://github.com/sgl-project/sglang/issues/43514

W5. `S32-04` — Long-prefill/decode interference: খাঁটি decode আর real agent সময় আলাদা

50k–130k reasoning agent-এ tool-call শেষ হলে নতুন turn আবার prefill শুরু করে; অন্য request তখন decode করছে। vLLM issue #54919-এ SM89 hardware-এ এই interference-এর concrete reproduction আছে—এটি **আমাদের জন্য test recipe**, SGLang-এর guaranteed bug নয়। SGLang-এ `--chunked-prefill-size` **2048/4096/8192** স্বতন্ত্রভাবে পরীক্ষা করুন।

- **D0:** সব resident decode; নতুন cold prefill নেই—engine decode ceiling।

- **D1:** 19 resident + 1 warm cached 90k—scheduler/cache overhead।
- **D2:** 19 resident + 1 cold 90k—decode starvation।
- **D3:** 16 resident + 4 cold 90k—interference scaling।
- **D4:** Real Duck/Poetiq tool-turn timeline—final sustained throughput।

Effective admission 20 না হলে actual running count ও queue/retraction time record করুন। `output tokens / active decode time` এবং `output tokens / full elapsed wall time (prefill/tools/waiting-সহ)` পৃথক। Smaller chunks interleaving উন্নত করলেও total prefill ধীর করতে পারে। GPU trace-এ GDN/attention/draft/collective critical path আলাদা করুন।

Memory rule: FP8 KV+TP4 full attention আনুমানিক **8192 bytes/token/GPU**; 24×130k সম্পূর্ণ স্বতন্ত্র resident histories-এর memory requirement অনেক। কিন্তু token-identical shared prefixes, cache tree ও logical worker queue ব্যবহার করে বেশি agent active রাখা যায়। এখানেই scheduling innovation করতে হবে; 1500+ টার্গেট কমানো নয়।

https://github.com/vllm-project/vllm/issues/54919

https://github.com/sgl-project/sglang/blob/main/docs/cookbook/autoregressive/Qwen/Qwen3.8-27B.mdx

W6. `S32-05/09` — NCCL TP4 critical path, SHM protocol ও no-P2P source-patch পরীক্ষা

আপনার report-এ একটি 116ms SGLang step-এর প্রায় **50.9ms all-reduce ও 8.1ms all-gather**। এগুলো cumulative durations; Nsight-এ overlap থাকলে 59ms-কে পুরো non-overlapped bottleneck ধরে লাভের হিসাব করা ভুল। প্রথমে timeline profiler দিয়ে critical-path slice বের করুন। তারপর NCCL tuning **একবারে একটি variable**:

Arm	Configuration delta	কী যাচাই করবেন
N1–N4	`NCCL_MIN_CTAS` / `NCCL_MAX_CTAS` = 1/2/4/8, supported হলে	Collective time ও GDN SM contention
N5/N6	`NCCL_ALGO=Ring` বনাম `Tree`	TP4 topology ও message-size sensitivity
N7/N8	`NCCL_PROTO=Simple` বনাম `LL`, installed NCCL support হলে	Small latency বনাম bandwidth
N9	`NCCL_SHM_USE_CUDA_MEMCPY`, support যাচাই করে	Host-staging/CPU synchronization
N10	Same collective under graph capture/on-demand	Captured TP overhead
N11	SGLang PR #39605 no-P2P staged collective source port	আগের PCIe-IPC-এর চেয়ে আলাদা algorithm

Model step-এ ব্যবহৃত **actual collective byte sizes** দিয়ে `nccl-tests` চালিয়ে এরপর একই 8k/40k/90k SGLang arm benchmark করুন। `NCCL_DEBUG=INFO` ও profiler-এ configured algorithm সত্যিই নির্বাচিত কি না দেখুন। কোনো microbenchmark দ্রুত হলেও decode TPS নেমে গেলে reject। আগের PCIe-IPC path 0.4× হওয়াকে নতুন staged host-copy algorithm নিষিদ্ধ করার কারণ বানাবেন না—কোড আলাদা হলে আলাদা experiment।

https://github.com/NVIDIA/nccl-tests

https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/env.html

https://github.com/sgl-project/sglang/pull/39605

W7. `S32-06` — Radix cache split করলে `mamba_value=None` ও 0-hit

SGLang #22935-এ অত্যন্ত নির্দিষ্ট code path আছে। `Req.init_next_round_input()` next-token logits তৈরির জন্য fresh-prefix match সর্বোচ্চ `input_len - 1` পর্যন্ত নিতে পারে। যদি ওই match compressed radix edge-এর মাঝখানে থামে, `_split_node()` একটি intermediate node বানায় যার `mamba_value=None`। এরপর `_match_post_processor()` valid-looking KV prefix-ও **zero cached tokens**-এ truncate করতে পারে। **একই prompt repeat করলেও** এই path তৈরি হতে পারে। এটি #43567 stale-state donation থেকে আলাদা bug।

Reproduction: 50k frozen prefix cache-এ insert করুন; দ্বিতীয় request-এ exact repeat; পরেরগুলোতে শেষ **1,16,64,512 tokens** বদলান এবং tool-call branch বানান। `match_prefix`, edge split, `mamba_value`, pre/post truncated length, cached tokens, forced re-tracking cost, cold vs warm next-logit KL ও actual wall-time collect করুন। Direct fix হিসেবে checkpoint ছাড়া KV match গ্রহণ করবেন না—এতে recurrent state ভুল হতে পারে। Valid intermediate checkpoint অথবা safe re-track-এর source change ও regression আগে পরীক্ষিত হতে হবে।

https://github.com/sgl-project/sglang/issues/22935

https://github.com/sgl-project/sglang/issues/22326

W8. `S32-07` — SGLang-এর leaderboard accuracy-gap কে engine bug থেকে আলাদা করা

SGLang DFlash6 দ্রুত হলেও 6–7 LB point-এর gap **শুধু engine-এর কারণে**—এমন প্রমাণ নেই। একই model prompt template, tokenizer, tool schema, seeds, agent strategy, deadlines, budgets ও task order দিয়ে paired পরীক্ষা করুন:

Arm	Engine ও configuration	আলাদা করে কী বোঝা যাবে
A	vLLM 0.32 FP8 target-only	FP8 numerical baseline
B	vLLM 0.32 INT4 target-only	Quantization effect
C	SGLang 0.5.21 INT4 target-only	Engine INT4 kernel/numerics
D	SGLang 0.5.21 INT4 + DFlash6	Draft verify/state effect
E	SGLang 0.5.21 INT4 + DFlash4	Width-specific quality/performance

Minimum: greedy first high-margin difference, ≥30k state-continuation, reject/grammar boundary, tool-call validity এবং **paired ARC task solve rate**। Temperature1 stochastic output exact match বাধ্যতামূলক নয়; task-level correct answers ও coverage আলাদা দেখান। SGLang upstream issue #42917-এর প্রথম INT4 PPL regression পরে লেখকের follow-up-এ stale CUDA **driver** cache-এর সঙ্গে যুক্ত হয়েছে—তাই প্রতিটি GPU launcher-এ import/CUDA init-এর আগে fresh `CUDA_CACHE_PATH` রাখুন, source SHA log করুন। Clean cache মানেই DFlash/#43567 correctness certified নয়।

https://github.com/sgl-project/sglang/issues/42917

https://github.com/sgl-project/sglang/pull/43567

https://github.com/sgl-project/sglang/issues/35150

W9. `S32-10` — অন্য GPU/engine-এর বাস্তব implementation থেকে নতুন SGLang patches

Code-mining candidate: **uniform multi-token CUDA graphs, FlashInfer decode-plan cache, no-sync draft proposal, GDN `in_proj_ba` split-K, CPU-resident embeddings, Mamba checkpoint retention, RecoverSSM এবং adaptive DFlash width**। Merlin/EAGLE implementation থেকে প্রত্যেকটির source diff, tensor shapes, graph-capture invariant, backend requirements ও measured microbenchmark সংগ্রহ করুন। SGLang-এর profile-এ যেটি বড় critical path কমাতে পারবে সেটি আগে port; patchগুলো একসঙ্গে merge করলে causal gain হারিয়ে যাবে।

EAGLE-4 drafter-এর জন্য extra GPU RAM, 24GB slot pressure, sm_89 kernel, 130k memory, speculative state equivalence ও license পরীক্ষা দরকার; এটিকে high-upside R&D হিসেবে রাখুন। ReplaySSM-তে `no_buffer` বাধ্যতামূলক হলে multi-turn prefix reuse ভাঙতে পারে, তাই source-specific negative test ছাড়া enable নয়। অন্য GPU-তে সফল ফল **পরীক্ষা বানানোর প্রেরণা ও code guide**, final performance prediction নয়।

https://github.com/halt95/merlin-engine

https://github.com/eagle-hq/EAGLE-4

https://huggingface.co/EAGLE-Corp/Qwen3.8-27B-EAGLE4

https://github.com/sgl-project/sglang/issues/37834

X. Version 3 — দ্বিতীয় AI-এর findings ও source coverage audit

এখানে আগত AI-এর **দুটি গবেষণা অংশ + URL appendix** আগের V1/V2-এর সঙ্গে মিলিয়ে coverage পরীক্ষা করা হয়েছে। নিচে item-wise mapping; কোনো ফল শুধু summary হিসেবে রেখে code/test বাদ দেওয়া হয়নি।

নতুন insight	সংশ্লিষ্ট section	Audit result
DFlash width4×S20 missing benchmark cell	W2	**INCLUDED**
#36889 Mamba pool silent cap ও +75% reported example	W3	**INCLUDED**
#41736 cross-request grouped-conv state contamination	W4	**INCLUDED**, sentinel test
#41737 all-NaN selector bounds	W4	**INCLUDED**, negative test
#43029 speculative committed-length boundary	W4	**INCLUDED**, grammar truncation
#36548/#38009 DFlash wrong-token examples	W4/W8	**INCLUDED**, target-only comparison
SM89 long-prefill starvation #54919	W5	**INCLUDED**, resident/cold/warm
2048/4096/8192 prefill chunk	W5	**INCLUDED**, equal work

নতুন insight	সংশ্লিষ্ট section	Audit result
#39605 no-P2P staged communication vs measured PCIe-IPC failure	W6	**INCLUDED**, distinct source algorithm
NCCL CTA/Tree/Ring/LL/Simple/SHM memcpy	W6	**INCLUDED**, microbench + live engine
#22935 radix split produces 0-hit	W7	**INCLUDED**, exact trigger
Mamba extra_buffer_lazy & resolved slot cap	W3	**INCLUDED**, allocation observability
FP8/INT4 + DFlash factorized oracle, leaderboard coverage	W8	**INCLUDED**
#42917 stale CUDA cache original/final explanation	W8 + prior N1/N2	**INCLUDED**, no old incorrect diagnosis
Merlin/EAGLE kernel designs এবং porting	W9 + previous sections	**INCLUDED**, actual implementation route
vLLM-only local argmax, FULL graph ও Ada FP8	অন্য vLLM v3 PDF	**ENGINE-SPECIFIC**, not omitted
130k independent/shared KV geometry	W5	**INCLUDED**, capacity engineering
Full primary URLs	W3–W9 + previous source pages	**INCLUDED**, clickable URLs

প্রতি আপডেটে দুই ধাপের বাধ্যতামূলক audit: প্রথমে আগের প্রতিটি heading, experiment, raw URL ও নতুন incoming AI finding audit; পরে final PDF-এর বাংলা text rendering, page clipping, clickable annotations, proper version label, source links ও table alignment পরীক্ষা। কোনো গুরুত্বপূর্ণ insight ছোট summary দিয়ে হারিয়ে ফেলব না।

Y. Version 4 — তৃতীয় AI গবেষণা সমন্বয়: SGLang 0.5.21 / Qwen3.8-27B

এই অংশে: `Pasted text(20261029-065936).txt`-এর **দুইটি গবেষণা** থেকে Franzen/Duck checkpoint LRU, SGLang #40865 Mamba retreat, 50k–130k multi-turn residency, SM89 low-M verification kernel routing, NCCL SHM এবং context-aware scheduling-এর পরীক্ষা যুক্ত হলো। আগের v1–v3 সব findings, raw URLs ও correctness-fix studies সংরক্ষিত। **1500+ tok/s** আমাদের main sustained aggregate accepted-output target; অন্য GPU-র source থেকে যত বেশি extractable implementation idea পাওয়া যায়, তত ভালো।

Y1. Updated priority campaign — SGLang-এর প্রথম ১৬টি পরীক্ষা

Rank	ID	Experiment	Decision / stop
P0	S4-00	12-hour real-agent event timeline/three TPS metrics	CPU only; missing evidence ! = PASS
P0	S4-01	INT4 target-only বনাম DFlash6, source-aware GDN correctness	same-weight oracle, ARC task-level paired

Rank	ID	Experiment	Decision / stop
P0	S4-02	#43567/#43559 + #43029/ #41736/#41737 causal audit	source guards + forced-reject/restore
P0	S4-03	DFlash W4 S20 (vs W4 S16, W6 S20)	cheap, exact prompts; no new wheel
P1	S4-04	actual TP4 allreduce/ allgather critical-path profile	Nsight + NCCL SHM shape match
P1	S4-05	NCCL ring/tree + CTA 2/4/8 + protocol	fresh process, real decode gains
P1	S4-06	checkpoint retreat & strategic retention (Franzen method)	GDN matching depth, KL, slot leaks
P1	S4-07	lazy Mamba 80/84/88/92 slots /20 streams	real running N vs KV competition
P1	S4-08	2048/4096/8192 mixed prefill chunk	decode starvation vs total prefill
P1	S4-09	cache-aware active session/ tool-return lease	retained-prefix, fairness, correct tasks
P1	S4-10	12/16/18/20/22/24 concurrency & 8k/40k/90k	do not relabel requested as resident
P2	S4-11	low-M GEMM/GDN in_proj_ba kernel routing	hot spot only; verify against target
P2	S4-12	DFlash verify kernel dispatch width 2/4/6/8	sm_89-specific CUDA profile
P2	S4-13	prefix radix split last-1/16/64/512 changes	cache hit and Mamba checkpoint integrity
P3	S4-14	PP4 or Merlin/Franzen staged fork	isolated source/binary/ quality gate
P3	S4-15	ReplaySSM with real prefix retention	otherwise no_buffer cache loss NO-GO

P0/P1 এখানে **পরীক্ষার অগ্রাধিকার**, production-engine auto approval নয়। Source branches/quantization/ parallelism/drafter বদলানো যাবে—কিন্তু প্রতিটি arm-এর exact binary, numeric baseline ও independent target-only correctness oracle দিতে হবে।

Y2. SGLang #40865: Mamba/GDN checkpoint-retreat-এ GPU compute নষ্ট হওয়ার সরাসরি evidence

[UPSTREAM VERIFIED] #40865-এর 24 September 2026 real-server A/B-তে Qwen3.8 hybrid **93% requests** checkpoint retreat করেছে এবং প্রায় **30% prefill compute** অনাবশ্যক পুনরায় করতে হয়েছে। Maintainers' experiment-এ complex replay mechanism cost-effective হয়নি; তাদের recommendation **checkpoint placement/retention policy**, নতুন replay execution path নয়। এটি সরাসরি আমাদের multi-turn tool-return workload-এর জন্য একটি concrete design problem।

কী ঘটে: full-attention KV node cache-এ থাকলেও matching earlier Mamba/GDN state না থাকায় last compatible checkpoint-এ ফিরে যেতে হয় এবং ওই gap আবার forward pass করতে হয়। Prefix hit-কে শুধু `cached_tokens / prompt_tokens` দিয়ে মাপলে কেন retreat ঘটল বোঝা যায় না।

Instrument without correctness change first: `full_kv_match_len`, `mamba_checkpoint_len`, `effective_cached_len`, `retreated_tokens`, `state_cache_available/evictable`, `final_prefill_checkpoint_present`, `cached_generated_tokens`, and per-turn reuse after tool return। Corrrelate by task/request IDs, no global counters alone। 5× same task follow-up, 10–30k generated thinking per cycle, gradual 50k→130k context, pressure 8/16/20 streams।

Compare: existing `extra_buffer_lazy`; then **checkpoint-policy-only** patch: prefill-final checkpoint retention, touch/LRU priority after valid tool return, and strategically valuable intermediate anchors. Avoid permanent pinning of every checkpoint: Mamba pool pressure can lower active streams and steal KV memory. Exact checkpoint serialized length and recurrent state must match; any stale-restore high-margin token flip => **NO-GO**. Also check #43567 stale-state path and #43559 allocation crash on this code path.

Sources

https://github.com/sgl-project/sglang/issues/40865

https://github.com/sgl-project/sglang/pull/43567

https://github.com/sgl-project/sglang/pull/43559

Y3. Franzen Duck implementation mining — retained prefix, LRU refresh, larger trim blocks

[UPSTREAM] Franzen's SGLang-based Duck repository uses strategically retained prefill Mamba checkpoints, LRU refresh and larger history-trimming blocks in its agent workflow. These mechanisms matter even when inference kernel efficiency is unchanged: existing model state survives across brief Python tool calls and future requests avoid 100k re-prefill.

Three separate candidate patches: (1) `touch/checkpoint-LRU` when an active tool-return request will soon resume; (2) `retain final prefill checkpoint` for a limited lease/size; (3) `trim history in larger, checkpoint-aligned blocks` instead of frequent tiny changes that defeat caching. Franzen's specialized Pennyroyal/fork code must be adapted into the pinned v0.5.21 semantics; blindly copying code that modifies radix tree / state donation is unsafe.

Experiment S4-06/09: replay a recorded 50k task prefix with tool calls and a 90k continuation. Compare baseline LRU, bounded checkpoint-retention, policy + session lease, and policy + larger trimming. Hold **same effective task text/compute budget** where judging pure cache; large trimming changes semantic context, so it must be scored as a separate agent-policy/accuracy experiment, not cache-only A/B. Count evicted checkpoints, `retreated_tokens`, time spent in prefill, real agent TPS, and correct ARC outputs/hour. Stop on unfair task starvation, memory leak, stale GDN restore or task-quality loss.

https://github.com/da-fr/arc-agi-3-solution

https://github.com/da-fr/arc-agi-3-solution/blob/main/WRITEUP.md

Y4. L4 low-M kernel dispatch: convert other engines' successful method into SGLang/Marlin experiments

[UPSTREAM VERIFIED] A single-L4 Qwen3.8-27B/DFlash2 llama.cpp reproduction changed quantized verification kernel routing (`GGML_MMVQ_MAX=2`) and measured about **+16.2%** in one setup. This is an **engineering precedent**: the fastest batch-one kernel may be slower than quantized GEMM for effective verification M=2–8, and optimal dispatch changes with draft width and active streams. We are NOT copying a GGML variable into SGLang; we are identifying the equivalent Marlin/compressed-tensors decision boundary and profiling it.

Trace: all dense linear projections and GDN `in_proj_qkvz/in_proj_ba/out_proj`, target verification query rows, drafter BF16 GEMM, output-head and TP vocabulary gathering; kernel name, occupancy, end-to-end CUDA time. Bench target `M in [1,2,4,6,8,16,20]`, 8k/40k/90k contexts, SM89, same INT4 g128 checkpoint. Detect where matvec vs matmul dispatch crosses, then a single localized source patch/flag if a faster supported kernel exists. **Reject** if different rounding causes confident target-token changes, state corruption, memory pressure or worse wall tok/s.

Related source ideas: Merlin split-K `in_proj_ba`, no-host-sync MTP second draft step and FlashInfer decode-plan cache; a source-backed performance idea does not require the source GPU to match exactly. But capture actual kernel dimensions and CPU sync before porting. SM120-specific assembly must be replaced with legal sm_89 kernel; never package an incompatible cubin.

https://github.com/hanxiao/Qwen3.8-27B-UD-Q4_K_XL-L4

https://github.com/da-fr/arc-agi-3-solution

Y5. SGLang 90k resident streams — KV and Mamba state must be budgeted jointly

Your EMS 90k `W6 S20` saw **7 actual running**, while `W4 S16` saw **9**: a key memory/admission clue, not just weak decode kernels. S20 benchmark uses **84 Mamba slots**; production S16 about **68**. `extra_buffer_lazy` has nominal ~4 state slots/admitted request, but actual in-flight pending/saved/evictable state varies. Changing slots 80/84/88/92 affects available KV pool and may raise or lower truly running N.

Capacity sweep gate: `resolved max-running`, `actual running p10/p50/p90`, `max-total-tokens`, `mamba_available`, `mamba_evictable`, `KV_capacity`, `request_retractions`, `per-request context`. For each arm, pre-allocate expected 90k contexts with prefix-sharing and independent-prefix groups; do not call queueing 20 streams “20 running.” Crash/`Can not alloc mamba cache`, OOM, or repeated prefix recompute => discard candidate.

Proposed short concurrency sweep: 12 / 16 / 18 / 20 / 22 / 24 logical sessions; 8k and 40k speed first, 90k and 130k only within measured resident capacity. Metric `accepted output tokens / actual wall time` and p95 wait (not request count alone).

https://github.com/sgl-project/sglang/issues/36889

https://github.com/sgl-project/sglang/issues/37834

Y6. Agent lease scheduler: stay on valuable warm prefix without starving other puzzles

Mechanism: after a short tool call, resume the same task promptly, while the last useful checkpoint and KV still exist. A CPU-first scheduling simulation can be built from real saved task log timestamps: active tasks, tool-start/return, prefill token budget, last-touch, cache depth, solver progress. Test `lease_ms` 0/1000/3000/10000 as a CPU replay candidate, then two GPU arms. Every leased session is bounded; deadline drain must preempt affinity; a stalled/high-memory task cannot occupy scarce slots forever.

Fairness invariant: at equal token and wall budgets, no task may remain indefinitely unserved; "cache hit improves" does not certify ARC solves. Record queue-age distribution, wasted re-prefill, task coverage, score, and total correct output work/GPU-hour. Large history-trim changes require paired ARC validation because context semantics differ.

Y7. Mixed cold-prefill/decode starvation test — distinct from saturated decode

SGLang's 2048/4096/8192 chunk recommendation should be measured with **same total work** in four conditions:

1. Stable 20 warm resident decode requests.
2. 19 resident decode + 1 cold 90k prefix.
3. 16 resident decode + 4 cold 90k prefixes.
4. Staggered tool-return/cold-prefill arrivals, recorded from actual agent trace.

For each independent `--chunked-prefill-size` launch, record decode TPOT p95, output tok/s per wall sec, prefill completion, waiting count, Mamba checkpoint evictions, `retreated_tokens`, CUDA graph status, NCCL critical path, and correct task outputs. Small chunks can help latency fairness while making overall prefill slower; choose via full workload outcome. Never modify installed SGLang high-risk internals before obtaining these logs.

https://github.com/vllm-project/vllm/issues/54919

https://github.com/sgl-project/sglang/issues/40865

Y8. NCCL TP4 and verification: actual attainable speed vs hypothetical compute ceiling

Stored SGLang profile states 116ms total, 50.9ms all-reduce and 8.1ms all-gather. These may include overlaps: a per-kernel sum is **not automatically 59ms on the non-overlapped critical path**. Nsight device timeline first. Then replay precise message sizes (≈0.1–2MB sample range as source claims, actual distribution authoritative) through NVIDIA `nccl-tests`, and only then test fresh server launches for NCCL `Ring/Tree`, protocol `Simple/LL`, `NCCL_MIN_CTAS/MAX_CTAS 2/4/8`, optional supported `NCCL_NTHREADS`. Do not retry previously measured slow FlashInfer PCIe-IPC or force P2P.

Counterexample: microbench reduces all-reduce 20%, but effective accepted tokens/step or CUDA graph coverage falls; engine throughput can still decline. Keep only improvements to critical-path verification step, p95 and mixed-agent TPS.

https://github.com/NVIDIA/nccl-tests

https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/env.html

https://github.com/sgl-project/sglang/pull/39605

Y9. SGLang ReplaySSM trade-off and corrected conclusion

Direct issue `#37834` documents that `--enable-linear-replayssm` forced/required `no_buffer`, which attached checkpoint too far into generated continuation and damaged shared-prefix hit. One saturated report: 71.91% → 29.23% cache hit and 134.5 → 93.4 output tok/s. A different low-load report showed larger TTFT but flat output TPS because offered load was the limiting factor. This is a **measured trade-off**, not proof replay kernel is slow.

Research arm only: test an explicit `extra_buffer_lazy`-compatible cursor-reset/backport or alternative checkpoint writer with ReplaySSM, if pinned source and kernel fit. Track `replayssm_write_pos`, prefix cache, checkpoint length, allocation and full KL. Blind `no_buffer` ReplaySSM production ⇒ NO-GO for this multi-turn workload.

https://github.com/sgl-project/sglang/issues/37834

https://github.com/sgl-project/sglang/pull/28695

Y10. Quantization/precision & other GPU techniques as implementable source ideas

Current RedHat INT4 W4A16 g128 targets speed but its leaderboard gap vs FP8 must be isolated by identical agent harness and same-weight vLLM/SGLang target-only controls. New W4, TP1-fit model or draft quantization may free memory for **more actually resident** streams; test that as a new checkpoint arm with ARC quality qualification, not a reason to freeze existing weights. Another GPU's low-M GEMM/LM-head patch is high-value source even if binary must be recompiled for SM89.

No need to research 1500+ NVIDIA achievement proof per user scope. Current objective is measurable transfer: offline wheel/build viability, startup under ten minutes, maximum context, real active count, correct programs/hour, then experimental combination.

Y11. Three denominators and production-economics experiment

Use all three counters on SGLang:

- `saturated_decode_tps = accepted_tokens / stable_saturated_decode_seconds`
- `mixed_serving_tps = accepted_tokens / entire_server_window_seconds` (prefill+queues)
- `agent_e2e_tps = accepted_tokens / end_to_end_agent_seconds` (tools/finalization)

At the current 90k benchmark 364 tok/s, increasing prefix reuse can raise agent E2E throughput even if kernel decode TPS remains 364. A 1500+ objective requires **both** high decode capacity and real occupancy, so the diagnostic must not conflate them. The 240×64k=15.36M tokens/pass, two passes 30.72M; at true 1500 useful tok/s generation alone needs ≈5.69h before startup/tools. Treat this as token-budget arithmetic, not achieved runtime.

Every promotion requires target-only confident-token parity, no state donation/restore corruption, valid tool calls, no OOM or slot leaks, equal task budgets, and a paired ARC solve-rate non-regression. Faster but lower leaderboard score is not final success.

Z. v4 second audit — incoming AI insight-to-section ledger

Incoming AI finding	Where captured	Audit / evidence status
Franzen checkpoints / LRU, history trim, Duck code	Y3/Y6	[UPSTREAM], source-port experiment
SGLang #40865 93% retreat, 30% prefill compute	Y2	[UPSTREAM VERIFIED]
Existing #43567/#43559 stale-state & crash	Y2, old O and W	[LOCAL DIAGNOSTIC+UPSTREAM]
L4 16.2% GGML verification GEMM routing	Y4	[UPSTREAM VERIFIED], adapt kernel method
Batch M=1/4/8/16/20 profile vs context	Y4	[RESEARCH]
Cache-aware session leases and fairness	Y3/Y6	[RESEARCH], CPU replay first
12/16/18/20/22/24 streams & actual resident	Y5	[LOCAL+TEST]
2048/4096/8192 mixed decode/prefill	Y7	[TEST]
NCCL critical path vs summed kernel time	Y8	[LOCAL PROFILE; verify overlap]
ReplaySSM no_buffer regression #37834	Y9, old S-P3-01	[UPSTREAM VERIFIED]
Speculative acceptance / quality vs target-only	Y1/Y10, old D	[MANDATORY GATE]
Three throughput denominator and 12h economics	Y11	[METHOD/ARITHMETIC]
Qwen3.8-Flash-Next / alternate models	old P3 source ideas only; Qwen3.8-27B remains scope	[SCOPE]
NVIDIA Team 1500+ throughput target	cover/Y1/Y11	[USER TARGET]

Conflicts accounted for: (1) More Mamba checkpoints improve reuse but consume Mamba slots; report both, rather than assuming permanent retention is free. (2) L4 GGML kernel patch is an **implementation proof of mechanism**, not a SGLang CLI switch. (3) Franzen larger history trim alters context and may alter accuracy; cannot claim purely cache-neutral speedup. (4) Constant high concurrency is not always best at 90k: cache, KV and GDN capacity decide. (5) Both papers preserve earlier v3 source evidence, experiments, fixes and URL appendix; this version adds rather than replaces.

Final coverage and rendering audit: compare old v3 headings/URLs against v4 source, verify literal new URLs, confirmed clickable link annotations in final PDF, and render sampled old+new pages for Bengali/text overflow. This PDF update does **not** modify Kaggle notebooks or run GPU experiments.

AA. Final integration v5: Claude E22 + তিন GPT + স্বাধীন গবেষণার পূর্ণ সমন্বয়

Research scope: Qwen3.8-27B, Kaggle 4×L4, SGLang 0.5.21 **source/patchable serving track**; goal sustained accepted-output **1500+ tok/s with no material accuracy loss**। Reference engine SGLang INT4+DFlash W6, কিন্তু **W4/W5/W7, EAGLE, SSM state, TP/PP, weight pack, CUDA-graph policy, cache pool, kernels, communication transport—সব variable**। অন্য GPU-তে তৈরি implementation code/measurement-কে adapt করার উপাদান হিসেবে রাখব। Source statuses: `[LOCAL RAW]`, `[LOCAL REPORT]`, `[UPSTREAM]`, `[CLAUDE MODEL]`, `[PLAN]`, `[VERIFY]`। E22 notebook এখনও pre-GPU experimental design, নতুন measured winner নয়।

AA1. নতুন unified priority queue—প্রতিটিতে exact intervention ও STOP

Rank	ID	Experiment / implementation idea	Cheap measurement	GO / STOP
P0-01	`SG-AR-TRACE`	TP4 NCCL 140 × ~983KB and 3 × ~11.9MB gather; actual CUDA critical path	raw E14A 343.88µs SHM; Nsight W4/W6 16/20	measurable step-time saving
P0-02	`SG-W4-S20`	DFlash2 draft width **4**, client 20, current extra_buffer_lazy	W4 S16 vs W6 S16 vs W6 S20, 8k/40k/90k	accepted-output TPS↑, parity PASS
P0-03	`SG-TARGET-ORACLE`	INT4 target-only vs INT4 DFlash W4/W6 vs vLLM INT4 target-only and FP8 reference	frozen prompt token IDs + forced-reject + tool grammar	no recurrent-state corruption
P0-04	`SG-CTA`	default NCCL SHM vs CTAs 2/4/8, optional NTHREADS512, Ring/Tree/LL only if supported	graph-aware 983040B microbench then 40k agent	verify-step p95 ↓, no hang
P0-05	`SG-CAPACITY`	Mamba-state slots (current 84), `max-running-requests`21, client20	actual live slots, KV pool, fallback, retraction and 90k running count	capacity↑ without cache-state loss
P0-06	`SG-CACHE-CLEAN`	fresh CUDA driver cache and deterministic target-only PPL/logprob repeat	alternate isolated cache vs original, cold/warm	repeatable correctness, no stale kernels

Rank	ID	Experiment / implementation idea	Cheap measurement	GO / STOP
P1-01	SG-COMMITTED-CKPT	selective PR #43029 committed-length checkpoint boundary	grammar-truncated speculative commit, shared-prefix repeat	Mamba boundary agrees actual committed tokens
P1-02	SG-DRAFT-CONV	PR #41736 grouped-conv tap contamination; #41737 NaN handling	16/20 concurrent, all-NaN injected, rejected/accepted drafts	no cross-request bleed, no punctuation loops
P1-03	SG-E14-MAMBA	#43559 allocation fail and #43567 stale lazy checkpoint, before risky pool changes	forced allocation failure, other task state isolation	no crash/ghost checkpoint
P1-04	SG-RADIX-LRU	Franzen selective checkpoints + LRU refresh + #22935 radix split test	same 50k prefix, modify final1/16/64/512 tokens	hit rate↑ and re-prefill↓ without drift
P1-05	SG-PREFILL	2048/4096/8192 chunk sizes, stagger tool arrival, decode fairness	19 warm+1 cold; 16+4 cold; 20 staged	mixed-wall tokens/hour↑
P1-06	SG-W5-W7	W5/W7 × concurrency 16/20; draft acceptance/time	one knob change per fresh server	effective accepted-output tokens per step↑
P1-07	SG-LIVE-LEASE	cache-aware agent lease, adaptive admission and tail refill	event-log CPU simulator → GPU A/B, fixed task budgets	correct ARC queries/GPU-hour ↑
P2-01	SG-HSAR-BF16	custom graph-capturable 4-rank pinned-host staged all-reduce, PR #39605 TP2 ideas adapted	CPU correctness→GPU microbench→full DFlash	net step-time gain; same logits
P2-02	SG-HSAR-INT8	quantized collective payload behind independent numerical gate	high-margin next-token, 90k tool-turn parity	quality pass and real gain
P2-03	SG-LOWM-SPLITK	DFlash/Marlin dispatch, GDN `in_proj_ba` split-K, LM-head sharded sampling	exact verification-M kernel trace, CUDA graphs	actual 40k/90k speed gain
P2-04	SG-MAMBA-REPLAY	ReplaySSM / no_buffer vs extra_buffer_lazy; keep prefix restoration	10k/30k/60k generated state, cache hit, accepted output	no 30% warm-turn regression

Rank	ID	Experiment / implementation idea	Cheap measurement	GO / STOP
P3	SG-ARCH-RD	Alternate quantized checkpoint, EAGLE-4, TP1×4 tailored, PP when supported, custom code	startup<10min; 90k fit, strict score gate	no source/backend incompatibility

নিয়ম: P0 “পরীক্ষার অগ্রাধিকার”, production-safe certification নয়। প্রতিটি arm-এ same model hash, prompt IDs, tools, seed, output budget; change exactly one parameter; then combine only independently positive arms. Success final metric correct ARC outputs/task/GPU-hour; raw TPS supplementary।

AA2. SGLang 116ms step—এবার underlying conditions স্পষ্ট

Claude-এর `research/final_report_bn.md`-এ **E14A R2 report থেকে** যে per-operator profile এসেছে, সেটি SGLang 0.5.21 INT4+DFlash **W8, 12 streams**, ~5.9k history, **96 verification rows**, 20 decode steps-এর; production W6 S20 নয়। Cumulative trace (sum ≈116.1ms, wall ≈116.7ms):

Category	E14A ms/step	Trace note
NCCL all-reduce, ~140 ×983,040B	**50.9**	44% of total, ~364µs each
NCCL all-gather, 3 calls	**8.1**	7%, likely vocab/logit communication
Marlin INT4 GEMM, ~512 calls	25.3	22%; not “zero weight cost”
BF16 drafter/lm_head other GEMM	14.5	12.5%
GDN Triton	8.0	7%
attention/elementwise/sampling/norm	~9.1	remainder

Four 12-stream widths produced target-only 35.3ms, W4 74.5ms, W6 96.4ms, W8 114.0ms per step. Claude regression fit `T_step ≈ 26 ms + 0.94 ms × verification_rows` with ±3ms residual. **[CLAUDE MODEL]** This short-context 12-stream model is a powerful diagnosis tool but NOT a calibrated 90k W6 S20 predictor. If accepted tokens/step = 20×4.21 ≈84.2, 40k measured 536 tok/s implies 157.1ms step. To reach 1500 with unchanged acceptance and active count requires **~56.1ms step**; at only 7 active XL requests, required step ~19.6ms. Therefore collective time, verification rows, active residency, decode attention length and acceptance must improve jointly.

Raw NCCL independent cross-check: ZIP `engine_pack/measurements/E14A_nccl_bench.json` has `fake:false`; four GPUs, `torch 2.13+cu130`, NCCL2.29.7; graph-replay median per 983KB all-reduce: SHM **343.8848µs**, forced P2P **1880.5212µs**; `NCCL_SHM_USE_CUDA_MEMCPY=1` graph timeout and eager 502.8µs. `nvidia-smi topo -p2p` says peer access OK, but transport slow; do not assume NVLink/P2P absence or faster P2P. `NCCL_SHM_DISABLE=1` is not a speed recipe: it can remove the winning path.

AA3. Host-staged 4-rank all-reduce: largest source-level challenge

Relevant upstream idea SGLang PR #39605 no-P2P staged communication. It is an implementation starting point, mostly TP2; our target is **4-rank TP4**, graph-capturable fixed-size ~983KB messages at **140 collective calls per verification step**। Design candidate:

1. Each rank writes BF16 tensor to pinned/mapped host staging segments; CUDA events or graph-safe semaphore tracks readiness.
2. Two-shot or ring reduction on staged data, no CPU/GPU device-wide synchronization in hot path.
3. Send reduced tensor back to all GPU ranks, preserve NCCL bitwise or bounded numerical reduction contract.
4. `all_reduce` custom dispatch must support both graph capture and replay, stream ordering, mixed batches, tensor shape/contiguity, rank failure, timeout and teardown.
5. First standalone test 983,040B ×140 plus 11,919,360B gather: compare NCCL SUM oracle and wall latency across 4 ranks; only then DFlash integration.

Research targets, not benchmark results: Claude's bf16 host-staged collective +14–17% net, INT8 payload +30–35% speed forecast. INT8 reduction is quantized communication and may change target logits, so research GO but production NO-GO until forced-reject, long-context, tool, and paired ARC checks. `NCCL_MIN_CTAS/MAX_CTAS=2/4/8`, `NCCL_NTHREADS=512`, `NCCL_ALGO` and `NCCL_PROTO` must be **separate fresh server launches**; retaining env changes after communicator setup is not a valid A/B. No forced PCIe-IPC again unless underlying source implementation has changed.

https://github.com/sgl-project/sglang/pull/39605

https://github.com/NVIDIA/nccl-tests

https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/env.html

https://github.com/syv-ai/HyperQwen/issues/254

AA4. W4×20, slot-budget and actual concurrency—three coupled levers

EMS reports:

DFlash	Client streams	8k	40k	XL ~90k	40k step
W4	16	592	526	363, 9 running	100.9ms
W6	16	583	521	370, 8 running	127.3ms
W6	20	**597**	**536**	**364, 7 running**	157.1ms
W8	16	573	477	363, 7 running	149.9ms

W4 S20 is the most important missing arm. Shorter verification and fewer speculative-state rows may lower pressure, but accepted tokens decrease. `accepted_outputs/verify_step_wall` and `actual_active_count` together decide winner; W4 S24 is a separate capacity-gated experiment. `--max-running-requests=21`, `--max-mamba-cache-size=84` correspond to four nominal slots per running request for extra_buffer_lazy; therefore **blind 80-slot testing with 21 max running may violate an invariant**. Only vary slots after source derives required minimum and log confirms admission. Any

slot tuning must report KV pool shrink, Mamba pool utilization, exhausted slots and preemption/retraction. #36889 public case got +75% in another hybrid after correcting an analogous hidden state-slot cap; our uplift unknown.

Cheap W4 edit (against W6 s20 control):

```
--speculative-algorithm DFLASH \
--speculative-num-draft-tokens 4 \
--max-running-requests 21 \
--max-mamba-cache-size 84 \
--mamba-radix-cache-strategy extra_buffer_lazy
```

Preserve actual draft checkpoint, attention backend, `chunked_prefill_size=4096`, CUDA graph decode sizes 1–21 and FP8 KV. Observe whether changing width changes preallocation—never assume preserved capacity from launch flags alone.

https://github.com/sgl-project/sglang/issues/36889

https://github.com/sgl-project/sglang/blob/main/docs/cookbook/autoregressive/Qwen/Qwen3.8-27B.mdx

AA5. Accuracy gap: distinct root-cause candidates, exact controls

Our SGLang INT4+DFlash W6 in an earlier comparable Poetiq route scored ~6–7 LB points below an FP8 vLLM path; **no single root cause proven** (weights, sampling, harness, model runner, DFlash state, tool grammar, cache reuse). E14A/EMS “BOUNDARY PASS” is not complete 60k ARC-solve parity.

Primary upstream **#36548** on another Qwen3.8-27B run: DFlash W8 serial 0/100 wrong, concurrency8 111/304 wrong, target-only 0/304 wrong. Forced draft rejection alone diverged on 19/32 in another test; first differing value was selected-step SSM transition output despite equal earlier inputs/states. W4, FP32 state, graph-off mitigated to 37/304 **but did not fix**. BF16 lm_head export lowered error to 1/304 **mitigation, not root-cause correction**. Your RedHat checkpoint already leaves lm_head BF16, so blindly repacking it again is not a justified fix. Issue auto-closed due inactivity, NOT proven fixed.

Other correctness leads to discriminate:

- **#41736** DFlash grouped-convolution taps can contaminate across requests; inspect minimal patch vs 0.5.21.
- **#41737** all-NaN speculative scores and fallback; test forced NaN/all rejected proposals.
- **#43029** Mamba checkpoint boundary incorrectly based on verified rather than actually committed token count under tool/grammar-constrained truncation; test truncated batch output length.
- **#43559/#43567** Mamba cache allocation can crash or stale checkpoint is reused when lazy allocation fails; force allocation failure, ensure no ghost state.
- **#42917** stale CUDA driver compute cache once corrupted W4A16 perplexity: reported fix clean cache. Prefer **isolated fresh CUDA_CACHE_PATH** and print resolved cache path; do not delete user directory indiscriminately.

- **#38009** DFlash2 thinking-mode greedy divergence and **#43514** grammar/multi-chunk acceptance collapse to ！ strings: include both acceptance and actual content checks.
- **#35150** forced-reject target state drift: model-output mismatches may exist with clean target-only path.

Four-way oracle matrix: (1) vLLM0.32 FP8 target-only, (2) vLLM0.32 INT4 target-only, (3) SGLang INT4 target-only, (4) SGLang INT4 DFlash4/6; same input token IDs, tokenizer, KV dtype, state precision, temperature, max budget, prompt/tool parser, ARC tasks. Distinguish FP8→INT4 quantization divergence from engine INT4 differences and from speculative drift. Compare first high-margin token, GDN recurrent transition at first divergence, sampled program validity and paired solve rate. A cross-version first-token non-identity at low margin is not automatically corruption; systematic high-confidence wrong branch is.

https://github.com/sgl-project/sglang/issues/36548

https://github.com/sgl-project/sglang/issues/38009

https://github.com/sgl-project/sglang/issues/35150

https://github.com/sgl-project/sglang/pull/41736

https://github.com/sgl-project/sglang/pull/41737

https://github.com/sgl-project/sglang/pull/43029

https://github.com/sgl-project/sglang/pull/43559

https://github.com/sgl-project/sglang/pull/43567

https://github.com/sgl-project/sglang/issues/42917

https://github.com/sgl-project/sglang/issues/43514

AA6. Mamba checkpoints and tool turns: state value, retreat, radix-split, reuse

Franzen's Duck/SGLang fork retained **strategically useful prefill checkpoints**, refreshed LRU and used coarser history compaction. SGLang #40865 reported checkpoint retreat on **93% requests** and **~30% avoidable prefill compute** in another constrained hybrid environment. Consequence: keeping full-attention KV alone is insufficient; generated replies/tool turns need a matching recurrent checkpoint.

Test sequence: 50k token prefix exact repeat → final 1/16/64/512 token modification → same-prefix repeated branch after tool result → eviction pressure from a 70k independent task → restored continuation. Measure checkpoint identity/time, `mamba_state_slot`, radix node split, prompt `cached_tokens`, actual reprefill and large-margin logits. `#22935` documents radix split creating `mamba=None` and sudden zero-hit pattern. Cursor/state after refused DFlash draft must never be committed. `--enable-linear-replayssm` may force `no_buffer` and shrink hit rate (public #37834 71.9→29.2%; output throughput 134.5→93.4 in that test); do not count only kernel time. Changes to LRU/cache retention require memory budget plus Mamba allocator no-leak test.

Harness design: implement a bounded cache-affinity lease for a short Python tool interval, preserving fairness. Freeze task ordering rules and score-equivalent budgets; simulate replay CPU-first with real event timestamps, then GPU A/B. Avoid making only the easiest/cached tasks advance while other ARC tasks expire at deadline. Tail refill should keep high active count by opening other attempts/branches with exact provenance, while preventing duplicate submissions and over-budget branches. Request scheduling is **not** a zero-risk source change if it affects how long agents reason.

https://github.com/sgl-project/sglang/issues/40865

https://github.com/sgl-project/sglang/issues/22935

https://github.com/sgl-project/sglang/issues/37834

https://github.com/da-fr/arc-agi-3-solution

https://github.com/da-fr/arc-agi-3-solution/blob/main/WRITEUP.md

AA7. Low-M GEMM, draft overhead, CUDA graphs and GDN kernels

SGLang currently uses INT4 Marlin WNA16, GDN Triton (FlashInfer fused GDN SM90+ gated), full attention via FlashInfer, decode CUDA graphs 1–21, `disable-prefill-cuda-graph`. Claude E14A profile shows GEMM ~39.8ms combined, GDN ~8ms in 12-stream W8 scenario; all-reduce ~50.9ms. **Therefore full new GEMM rewrite should follow critical-path profiling, not precede NCCL sweep.**

Source ideas: Han Xiao's Qwen3.8 single-L4 llama.cpp speculative verification routed to wrong small-M GEMV, dispatch fix +16.2%; extract batch-dimension crossover principle, not llama.cpp binary. Franzen's low-M BF16 GEMM SM120 patch demands SM89 adaptation; Merlin GDN `in_proj_ba` split-K and no-sync MTP second-draft-step need Qwen3.8 W4A16 shape-specific benchmark. Width W4/5/6/7, actual accepted/step and target verification rows; SGLang EAGLE MTP can be a quality control (historical 478/402 vs DFlash 583/521) even if slower. ReplaySSM, GDN fp16/fp32 recurrent state changes and DFlash width affect arithmetic/state semantics → long-context precision gate. DFlash W8 had less throughput than W4/W6; don't repeat identical W8 unless kernel code changes.

https://github.com/hanxiao/Qwen3.8-27B-UD-Q4_K_XL-L4

https://github.com/da-fr/arc-agi-3-solution

https://github.com/sgl-project/sglang/issues/28511

https://github.com/sgl-project/sglang/pull/28695

AA8. E22 speedlab notebook—planned arms, deployment gates এবং সীমা

Claude ZIP's `e22_notebooks/Q27-XHIGH-E22_SPEEDLAB_SGLANG.ipynb` has 10 cells, 3+ matrix/engine/report cells. It plans W4/W6, streams 20/24, NCCL CTA8, larger prefill, combinations; attached `engine_pack/sglang/results/ems_report.md` is **prior baseline**, not fresh E22 outcome. No E22 completed per-arm GPU throughput artifact attached. A notebook builder file exists, but user requested research PDF update—not notebook execution or modification. For each future candidate use one fresh process, explicit `nvidia-smi`, `NCCL_DEBUG=INFO`, kernel selection, `max_running` and Mamba slots, SHA of generated notebook, offline wheel dependency list, startup ≤10min, source fallback, generated content gate.

Campaign:

```
S0  CPU: compare notebook/build hashes, installed flags & source patches, no GPU
S1  8k: W4/S20 vs W6/S20, NCCL SHM CTA2/4/8, verify step and emitted TPS
S2  40k: winning widths and matched 16/20, target-only vs DFlash correctness
S3  90k–130k: 5-turn tool trace, request admission, cache retreat and prefill
S4  custom host-staged AR proof, 983KB×140 and 11.9MB gather in graphs
S5  long ARC paired sample; no LB adoption if confidence/score regresses
```

Keep `P2P/IPC`, `NCCL_SHM_USE_CUDA_MEMCPY`, blind `no_buffer`, speculative-state unchecked optimizations disabled as measured/known risks. Out-of-scope model Flash-Next results and alternate GPU benchmarks remain **implementation-mining inputs**, not an invitation to switch current Qwen3.8-27B target.

AA9. Full source coverage audit: three GPT studies + our study + Claude E22

Source	Important insights and where carried forward
GPT-1 `Pasted text(9).txt`	Merlin alternative checkpoint and TP layouts, adaptive EAGLE-4, 16-attention KV formula, MTP-depth trade-off, speculative quality matrix: previous W/V, AA4/AA7
GPT-2 `Pasted text(10).txt`	#42917 cache PPL root correction, W4 S20, #43029 truncation, FP16/BF16 GDN eligibility, PR #60532 and SM89 kernels: previous V3, AA4/AA5/AA7
GPT-3 `Pasted text(20261029-065936).txt`	#36889 slot cap, #41736/#41737, #22935 radix split, PR #39605, AIMO-2 benchmarking, graph probes: previous V3/Y, AA2–AA7
Independent GPT Research v1–v4	Raw EMS/VMS matrix, ReplaySSM regression, throughput denominator, safe caches, cache-aware sessions, independent quality caveats: all v4 content intact plus AA1–AA8
Claude E22 (report + 5 agent notes + raw data)	116ms operator breakdown; 26+0.94×rows fit; real NCCL 343.88µs; host-staged 4rank BF16/INT8; tail occupancy; NVFP4/PP/graph feasibility; width+state-slot math: AA1–AA8 and sister vLLM v5 PDF

Conflicts and clarifications: Claude's `+14–35%` staged-AR estimates **not measured**; custom INT8 collective **not guaranteed accuracy-neutral**. 116ms operator breakdown is specifically **E14A W8 S12**; don't attribute it to all W6 S20 runs. Exact 1500 tok/s is research target not a demonstrated E22 speed. 4×L4 peer access actually **enabled** but forced P2P transport was 5.47× slower. Numerical parity for vLLM FP8 versus SGLang INT4 needs matched tasks and same harness; current LB gap cannot be assigned to any one kernel. New 0.5.22 correctness backports are test candidates, not present in stock 0.5.21 until verified.

Content preservation contract: all v4 experiment sections, headings and literal URL sources preserved; v5 additions have their own unique experiment IDs and STOP conditions. PDF-level audit must verify link annotations, Bengali fonts, page-boundaries and every key Claude/new-GPT finding. No engine notebook was modified or run for this document update.

Version 5.0 · এই PDF পুনরায় আপডেটযোগ্য; পরবর্তী AI গবেষণা যোগ করার সময়ে প্রমাণ, experiment ID ও revision history সংরক্ষণ করতে হবে।